Colab 1 · RSA: fundamentos, ataques y cifrado híbrido

Construiremos RSA, justificaremos su correctitud y estudiaremos cuatro ataques que muestran las condiciones necesarias para usarlo de forma segura. Después introduciremos RSA-OAEP y lo combinaremos con AES-GCM en una aplicación que envía mensajes y archivos entre un servidor y varios clientes.

Bloque	Propósito
1. De la idea a RSA	Distinguir evaluación pública, inversión privada y seguridad
2. Herramientas aritméticas	Calcular MCD, coeficientes de Bézout, inversos y potencias
3. Generación de claves	Obtener los factores y exponentes de RSA
4. RSA en funcionamiento	Aplicar las operaciones y demostrar su correctitud
5. RSA-CRT	Acelerar la inversión mediante cálculos módulo cada primo
6. Seguridad y ataques	Relacionar cada ataque con sus hipótesis y su alcance
7. RSA-OAEP	Pasar de una permutación de enteros a un esquema de cifrado
8. Aplicación final	Establecer una clave de sesión y proteger varios envíos

Ejecución. Se requieren congruencias, divisibilidad y Python básico. Ejecute las celdas en orden: las funciones y las claves se reutilizan. Los ejemplos generan sus propios datos. Los `assert` comprueban casos concretos; las explicaciones justifican las propiedades generales.

Alcance. La aritmética se implementa con fines docentes y no garantiza tiempo constante. OAEP y AES-GCM se usan mediante PyCryptodome. La aplicación TCP se ejecuta dentro del entorno de Colab y sirve para estudiar la integración de sus componentes.

0. Preparación

Usaremos **2048 bits por primo**: el módulo $n = pq$ tendrá 4095 o 4096 bits. Fijamos $e = 65537$ y 64 rondas de Miller–Rabin. La sección 3 explicará cómo intervienen estos parámetros al generar la clave principal.

Notación	Significado
pk, sk	Clave pública y privada; la generación devuelve `(pk, sk)`
x, y	Enteros de entrada y salida de la permutación RSA
M, C	Mensaje y cifrado como cadenas de bytes
$a \bmod N$	Resto en $\{0, \ldots, N-1\}$
$\leftarrow, \xleftarrow{\$}$	Asignación y muestreo uniforme
$\perp$	Rechazo; en Python se representa con una excepción o `None`

Las asignaciones de tuplas son simultáneas: todos los valores de la derecha se calculan antes de actualizar las variables. La siguiente celda prepara los parámetros y las herramientas para medir tiempos y mostrar enteros.

```
import secrets
from time import perf_counter
from statistics import median

BITS_PRIMO = 2048
EXPONENTE_PUBLICO = 65537
RONDAS_MR = 64

def mostrar_entero(nombre, valor):
    """Divide la impresión de un entero grande en líneas de 88 caracteres."""
    decimal = str(valor)
    print(nombre, ":", valor.bit_length(), "bits")
    for inicio in range(0, len(decimal), 88):
        print(decimal[inicio:inicio + 88])

print("Bits por primo:", BITS_PRIMO)
print("Exponente público:", EXPONENTE_PUBLICO)
print("Rondas de Miller-Rabin:", RONDAS_MR)
```

```
Bits por primo: 2048
Exponente público: 65537
Rondas de Miller-Rabin: 64
```

1. De la idea a RSA

1.1. Compartir un secreto sin enviarlo directamente

El cifrado simétrico exige que emisor y receptor conozcan una misma clave. El cifrado de clave pública permite proteger información para un receptor sin compartir previamente una clave secreta con él.

Alice genera una clave pública y una privada. Bob cifra con la **pública de Alice** y Alice descifra con su **privada**. En la aplicación final, Alice será el servidor y Bob representará a cada cliente.

Bob debe disponer de una copia auténtica de la clave pública: si un atacante la sustituye por la suya, recibirá información cifrada para él. Supondremos que esa autenticidad ya está establecida; una conexión TCP por sí sola no la proporciona.

1.2. Esquema trapdoor: tres algoritmos

Antes de construir el cifrado, estudiaremos su operación matemática básica. Un esquema trapdoor, o de trampa, $\mathrm{T} = (G, F, I)$ permite evaluar una función con información pública e invertirla usando información privada:

- **Generación:** $(pk, sk) \leftarrow G(1^\kappa)$.
- **Evaluación pública:** $y \leftarrow F(pk, x)$.
- **Inversión privada:** $x' \leftarrow I(sk, y)$.

κ es el parámetro de seguridad y 1^κ lo representa como una cadena de κ unos. La clave pública determina los dominios de entrada X_{pk} y salida Y_{pk}. Para toda entrada válida se exige

$$I(sk, F(pk, x)) = x.$$

Esta es la condición de **correctitud**. La **trampa** es la información privada que permite efectuar la inversión. En RSA, la función será una permutación: cada salida corresponderá a una única entrada.

1.3. Correctitud no es seguridad

La correctitud describe lo que puede hacer quien conoce la trampa. La **unidireccionalidad** exige que, con solo la clave pública, sea difícil recuperar una entrada elegida uniformemente al azar a partir de su imagen. El siguiente juego formaliza esa exigencia:

```
JuegoInversion(A, κ)
    (pk, sk) ← G(1^κ)
    x ←$ X_pk
    y ← F(pk, x)
    x_estimado ← A(pk, y)
    devolver [x_estimado = x]
```

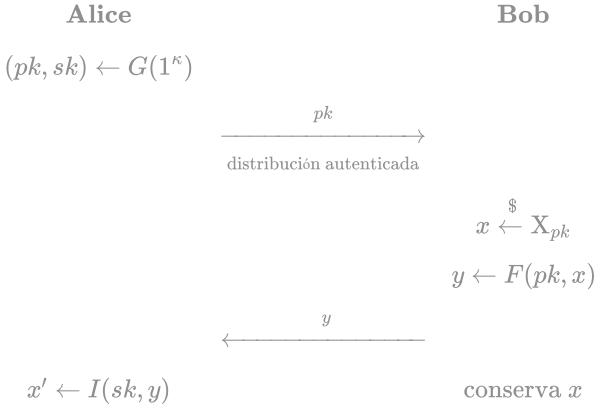
$$ \operatorname{Adv}^{\mathrm{OW}}_{\mathrm{T},A}(\kappa) =\Pr[x_{\mathrm{estimado}}=x]. $$$$ \operatorname{Adv}^{\mathrm{OW}}_{\mathrm{T},A}(\kappa) =\Pr[x_{\mathrm{estimado}}=x]. $$

Se pide que esta probabilidad sea despreciable para todo adversario de tiempo polinómico: para cada entero $c>0$$c>0$, debe ser menor que $\kappa^{-c}$$\kappa^{-c}$ cuando $\kappa$$\kappa$ sea suficientemente grande. La probabilidad incluye la generación de claves, la elección de la entrada y la aleatoriedad del adversario.

La distribución de la entrada importa. Si se sabe que es un dígito, basta evaluar públicamente las diez posibilidades y comparar. Por ello, la unidireccionalidad por sí sola no proporciona un cifrado seguro para mensajes predecibles.

1.4. Compartir una entrada mediante un esquema trapdoor

El siguiente intercambio muestra el papel de cada algoritmo. Bob elige la entrada y Alice la recupera mediante la inversión privada.



La correctitud garantiza $x' = x$. El observador dispone de pk e y. Este diagrama describe el intercambio básico; para obtener una clave de sesión con las propiedades necesarias usaremos RSA-OAEP y una función de derivación en el bloque 8.

1.5. Esquema trapdoor RSA

Elegimos primos impares distintos p, q y calculamos

$$n = pq, \qquad \varphi(n) = (p-1)(q-1).$$

$\varphi(n)$ cuenta los enteros de $\{1, \ldots, n-1\}$ coprimos con n. Elegimos e con $\gcd(e, \varphi(n)) = 1$ y calculamos

$$d = e^{-1} \bmod \varphi(n), \qquad ed \equiv 1 \pmod{\varphi(n)}.$$

La **clave pública** contiene (n, e). La **clave privada** permite usar d; también conservaremos p, q para acelerar la inversión con CRT. Ambos dominios son $\mathbb{Z}_n = \{0, \ldots, n-1\}$.

Conocer los factores permite obtener $\varphi(n)$ y calcular d a partir de e. Cuando fijamos e de antemano, exigimos $\gcd(e, p-1) = \gcd(e, q-1) = 1$ para garantizar que ese inverso exista.

1.6. Dos potencias modulares

La evaluación pública y la inversión privada son, respectivamente,

$$y = F_{\mathrm{RSA}}(pk, x) = x^e \bmod n, \qquad x' = I_{\mathrm{RSA}}(sk, y) = y^d \bmod n.$$

Demostraremos en el bloque 4 que $x' = x$ para toda entrada de $\mathbb{Z}_n$. Estas operaciones actúan sobre enteros y son deterministas. El tratamiento de mensajes en bytes y la aleatoriedad del cifrado se incorporarán con OAEP.

Para implementarlas necesitamos MCD, inversos y potencias modulares. Después utilizaremos Miller–Rabin para buscar los factores primos. El exponente $e = 65537 = 2^{16} + 1$ tiene solo dos bits iguales a uno, lo que reduce el trabajo de la evaluación pública.

2. Herramientas aritméticas

2.1. Euclides: máximo común divisor

El MCD permite comprobar coprimalidad y, más adelante, recuperar factores en dos ataques. Euclides utiliza la identidad

$$ \gcd(a,b)=\gcd(b,a\bmod b),\qquad b\ne0. $$$$ \gcd(a,b)=\gcd(b,a\bmod b),\qquad b\ne0. $$

Se sustituye el par por el divisor y el resto hasta que el segundo valor sea cero. El primero es entonces el MCD: $(252,105)\to(105,42)\to(42,21)\to(21,0)$$(252,105)\to(105,42)\to(42,21)\to(21,0)$.

```
MCD(a, b)
    (u, v) ← (|a|, |b|)
    mientras v ≠ 0:
        (u, v) ← (v, u mod v)
    devolver u
```

Adoptamos $\gcd(0,0)=0$$\gcd(0,0)=0$. Para entradas positivas se realizan $O(1+\log\min(a,b))$$O(1+\log\min(a,b))$ divisiones; su costo en bits depende del tamaño de los operandos.

La primera prueba cuenta los coprimos con $11\cdot17$$11\cdot17$ y obtiene $\varphi(187)=160$$\varphi(187)=160$. Las siguientes comprueban el ejemplo de Euclides y un caso grande en el que $B$$B$ divide exactamente a $A$$A$.

```
    def mcd(a, b):
        """Devuelve gcd(a,b) >= 0; por convenio, gcd(0,0)=0."""
        u, v = abs(a), abs(b)
        while v != 0:
            u, v = v, u % v
        return u
```

```
    # Contar los enteros coprimos con n permite obtener phi(n) en este ejemplo pequeño.
    n_ejemplo = 11 * 17
    phi_ejemplo = 0
    for entero in range(1, n_ejemplo):
        if mcd(entero, n_ejemplo) == 1:
            phi_ejemplo += 1

    print("n =", n_ejemplo)
    print("phi(n) =", phi_ejemplo)
    print("e = 7 y phi(n) son coprimos:", mcd(7, phi_ejemplo) == 1)
```

```
    n = 187
    phi(n) = 160
    e = 7 y phi(n) son coprimos: True
```

```
    print("MCD(252, 105) =", mcd(252, 105))

    # A = B * (2**1024 + 1), de modo que B divide exactamente a A.
    A_mcd = 2**2048 - 1
    B_mcd = 2**1024 - 1
    print("El MCD de los enteros grandes coincide con B:", mcd(A_mcd, B_mcd) == B_mcd)
```

```
    MCD(252, 105) = 21
    El MCD de los enteros grandes coincide con B: True
```

2.2. Euclides extendido

Euclides extendido calcula el MCD junto con coeficientes de Bézout: para $a,b\ge0$$a,b\ge0$, devuelve $(g,s,t)$$(g,s,t)$ tales que

$$ g=\gcd(a,b)=sa+tb. $$$$ g=\gcd(a,b)=sa+tb. $$

Mantiene cada resto como una combinación de las entradas: $r_i=s_i a+t_i b$$r_i=s_i a+t_i b$. Si el cociente de la división es $h$$h$, la actualización del resto exige actualizar sus coeficientes de la misma manera:

$$ r_0-hr_1=(s_0-hs_1)a+(t_0-ht_1)b. $$$$ r_0-hr_1=(s_0-hs_1)a+(t_0-ht_1)b. $$

```
EuclidesExtendido(a, b)
    requiere a ≥ 0 y b ≥ 0
    (r0, r1) ← (a, b)
    (s0, s1) ← (1, 0)
    (t0, t1) ← (0, 1)
    mientras r1 ≠ 0:
        h ← piso(r0 / r1)
        (r0, r1) ← (r1, r0 - h·r1)
        (s0, s1) ← (s1, s0 - h·s1)
        (t0, t1) ← (t1, t0 - h·t1)
    devolver (r0, s0, t0)
```

Con $a = 7, b = 160$, el algoritmo obtiene $(g, s, t) = (1, 23, -1)$:

$$1 = 23 \cdot 7 - 160.$$

Módulo 160, esto significa $23 \cdot 7 \equiv 1 \pmod{160}$: el coeficiente 23 proporciona el inverso que necesitaremos en la siguiente sección. `mostrar_pasos=True` permite seguir los restos y sus coeficientes.

```
    def euclides_extendido(a, b, mostrar_pasos=False):
        """Devuelve (g, s, t), donde g = MCD(a, b) = s*a + t*b."""
        if a < 0 or b < 0:
            raise ValueError("Las entradas deben ser no negativas.")

        r0, r1 = a, b
        s0, s1 = 1, 0
        t0, t1 = 0, 1

        if mostrar_pasos:
            print("Restos:", r0, r1, "| Coeficientes de a:", s0, s1,
                  "| Coeficientes de b:", t0, t1)

        while r1 != 0:
            h = r0 // r1
            r0, r1 = r1, r0 - h * r1
            s0, s1 = s1, s0 - h * s1
            t0, t1 = t1, t0 - h * t1

            if mostrar_pasos:
                print("Restos:", r0, r1, "| Coeficientes de a:", s0, s1,
                      "| Coeficientes de b:", t0, t1)

        return r0, s0, t0
```

```
    g, s, t = euclides_extendido(7, 160, mostrar_pasos=True)
    print("Resultado (g, s, t):", (g, s, t))
    print("Comprobación: s*7 + t*160 =", s * 7 + t * 160)
```

```
    Restos: 7 160 | Coeficientes de a: 1 0 | Coeficientes de b: 0 1
    Restos: 160 7 | Coeficientes de a: 0 1 | Coeficientes de b: 1 0
    Restos: 7 6 | Coeficientes de a: 1 -22 | Coeficientes de b: 0 1
    Restos: 6 1 | Coeficientes de a: -22 23 | Coeficientes de b: 1 -1
    Restos: 1 0 | Coeficientes de a: 23 -160 | Coeficientes de b: -1 7
    Resultado (g, s, t): (1, 23, -1)
    Comprobación: s*7 + t*160 = 1
```

2.3. Inverso modular

Un inverso de $a$$a$ módulo $N$$N$ es un valor $b$$b$ tal que $ab\equiv1\pmod N$$ab\equiv1\pmod N$. Existe exactamente cuando $\gcd(a,N)=1$$\gcd(a,N)=1$. En ese caso, Bézout proporciona $sa+tN=1$$sa+tN=1$ y, al reducir módulo $N$$N$, obtenemos el inverso $s\bmod N$$s\bmod N$.

```
InversoModular(a, N)
    requiere N ≥ 2
    (g, s, t) ← EuclidesExtendido(a mod N, N)
    si g ≠ 1:
```

```
        rechazar: el inverso no existe
    devolver s mod N
```

Lo usaremos para calcular $d$$d$ y para recombinar los resultados de CRT. Las pruebas recuperan $7^{-1}\bmod160=23$$7^{-1}\bmod160=23$ y el inverso de 65537 módulo $2^{2048}$$2^{2048}$. Este último existe porque 65537 es impar. En cambio, 2 comparte un factor con ese módulo, por lo que la función lanza una excepción.

```
def inverso_modular(a, modulo):
    """Devuelve a^(-1) mod modulo o rechaza si el inverso no existe."""
    if modulo < 2:
        raise ValueError("El módulo debe ser al menos 2.")
    g, s, _ = euclides_extendido(a % modulo, modulo)
    if g != 1:
        raise ValueError("El inverso no existe: el MCD es distinto de uno.")
    return s % modulo
```

```
inverso = inverso_modular(7, 160)
print("Inverso de 7 módulo 160:", inverso)
print("Comprobación:", 7 * inverso % 160)

modulo_grande = 2**2048
inverso_grande = inverso_modular(EXPONENTE_PUBLICO, modulo_grande)
print("Comprobación con enteros grandes:",
      EXPONENTE_PUBLICO * inverso_grande % modulo_grande)

try:
    inverso_modular(2, modulo_grande)
except ValueError as error:
    print("Caso sin inverso:", error)
```

```
Inverso de 7 módulo 160: 23
Comprobación: 1
Comprobación con enteros grandes: 1
Caso sin inverso: El inverso no existe: el MCD es distinto de uno.
```

2.4. Exponenciación modular

La operación central de RSA y Miller–Rabin es $a^b\bmod N$$a^b\bmod N$. Para calcularla sin construir $a^b$$a^b$, usamos la representación binaria del exponente. Por ejemplo, $13=8+4+1$$13=8+4+1$ implica $a^{13}=a^8a^4a$$a^{13}=a^8a^4a$.

El algoritmo recorre los bits de $b$$b$ desde el menos significativo. La variable $u$$u$ toma las potencias $a,a^2,a^4,\ldots$$a,a^2,a^4,\ldots$ módulo $N$$N$; cuando el bit correspondiente es uno, multiplica $u$$u$ por el acumulador $r$$r$. Dividir $v$$v$ por dos descarta el bit ya procesado.

```
PotenciaModular(a, b, N)
    requiere b ≥ 0 y N ≥ 2
    r ← 1
    u ← a mod N
    v ← b
    mientras v > 0:
        si v es impar:
            r ← r·u mod N
        u ← u·u mod N
        v ← piso(v / 2)
    devolver r
```

Cada operación se reduce módulo N, de modo que sus operandos permanecen entre 0 y $N-1$ y los productos son menores que N^2. Para $b>0$ se realizan $O(\log b)$ multiplicaciones modulares; si $b=0$, se devuelve uno.

Con $e=65537$, este código hace 17 cuadrados y dos multiplicaciones en el acumulador. Las primeras pruebas calculan $2^{17}\bmod 91$ y muestran $16^2\bmod 91$, uno de los cuadrados intermedios. Después se usa $3^{13}\bmod 17$ y se contrasta una operación grande con `pow(a,b,N)`. `pow` sirve como referencia independiente.

```
def potencia_modular(base, exponente, modulo):
    """Cuadrados y multiplicaciones; exponente >= 0 y modulo >= 2."""
```

```
    if exponente < 0 or modulo < 2:
        raise ValueError("Se requieren exponente >= 0 y módulo >= 2.")
    r = 1
    u = base % modulo
    v = exponente
    while v > 0:
        if v % 2 == 1:
            r = r * u % modulo
        u = u * u % modulo
        v //= 2
    return r
```

```
print(potencia_modular(2, 17, 91))
```

```
32
```

```
16*16 % 91
```

```
74
```

```
print("3**13 mod 17 =", potencia_modular(3, 13, 17))

# pow ofrece una comprobación independiente de nuestra implementación.
modulo_potencia = 2**2048 - 159
base_potencia = secrets.randbelow(modulo_potencia)
exponente_potencia = 2**2047 + 65537
resultado = potencia_modular(base_potencia, exponente_potencia, modulo_potencia)
referencia = pow(base_potencia, exponente_potencia, modulo_potencia)

print("El resultado coincide con pow:", resultado == referencia)
print("Bits de e:", bin(EXPONENTE_PUBLICO))
assert resultado == referencia
```

```
3**13 mod 17 = 12
El resultado coincide con pow: True
Bits de e: 0b10000000000000001
```

3. Generación de claves

3.1. De Fermat a Miller–Rabin

Con las herramientas aritméticas disponibles, falta obtener los factores primos. El pequeño teorema de Fermat establece que, si p es primo y $a \not\equiv 0 \pmod{p}$, entonces

$$a^{p-1} \equiv 1 \pmod{p}.$$

Incumplir esta igualdad demuestra que el candidato es compuesto. Cumplirla para una base no certifica primalidad. Miller–Rabin refuerza la comprobación examinando los cuadrados intermedios.

La siguiente celda parte de 3 módulo 17 y calcula sucesivamente $3^2, 3^4, 3^8$, cuyos residuos son $9, 13, 16$. El último es -1 módulo 17; su siguiente cuadrado sería uno. Este comportamiento explica la condición de aceptación que estudiaremos a continuación.

```
a=3
N=17

a=potencia_modular(a, 2, N)
print(a)
a=potencia_modular(a, 2, N)
print(a)

a=potencia_modular(a, 2, N)
print(a)
```

```
9
13
16
```

3.2. Una ronda de Miller–Rabin

Para un candidato impar $N\ge5$$N\ge5$, escribimos

$$ N-1=2^s u,\qquad u\text{ impar}, $$$$ N-1=2^s u,\qquad u\text{ impar}, $$

elegimos una base $2\le a\le N-2$$2\le a\le N-2$ y calculamos

$$ z_0=a^u\bmod N,\qquad z_{j+1}=z_j^2\bmod N. $$$$ z_0=a^u\bmod N,\qquad z_{j+1}=z_j^2\bmod N. $$

Así, $z_j=a^{2^j u}\bmod N$$z_j=a^{2^j u}\bmod N$ y $z_s=a^{N-1}\bmod N$$z_s=a^{N-1}\bmod N$. Si $N$$N$ es primo, Fermat garantiza $z_s=1$$z_s=1$. Además, las únicas raíces cuadradas de uno módulo un primo son $1$$1$ y $-1$$-1$.

Por tanto, si $z_0$$z_0$ no es uno, el valor inmediatamente anterior a la primera aparición de uno debe ser $N-1$$N-1$, que representa $-1$$-1$. La base **pasa la ronda** precisamente cuando

$$ z_0=1 \quad\text{o bien}\quad z_j=N-1\text{ para algún }j\in\{0,\ldots,s-1\}. $$$$ z_0=1 \quad\text{o bien}\quad z_j=N-1\text{ para algún }j\in\{0,\ldots,s-1\}. $$

Solo se examina hasta $z_{s-1}$$z_{s-1}$: incluso si el primer uno apareciera en $z_s$$z_s$, antes tendría que aparecer $N-1$$N-1$.

Rechazo dentro del bucle. Si un cuadrado produce uno sin haber encontrado $N-1$$N-1$, su predecesor $w$$w$ cumple

$$ w^2\equiv1\pmod N,\qquad w\not\equiv1\pmod N,\qquad w\not\equiv-1\pmod N. $$$$ w^2\equiv1\pmod N,\qquad w\not\equiv1\pmod N,\qquad w\not\equiv-1\pmod N. $$

Esta raíz no trivial de uno demuestra que $N$$N$ es compuesto. Por ejemplo, con $N=21,a=8$$N=21,a=8$:

$$ 20=2^2\cdot5,\qquad z_0=8^5\bmod21=8,\qquad z_1=8^2\bmod21=1. $$$$ 20=2^2\cdot5,\qquad z_0=8^5\bmod21=8,\qquad z_1=8^2\bmod21=1. $$

También se rechaza si se agotan los $s-1$$s-1$ cuadrados sin encontrar $N-1$$N-1$. En ambos casos, la base es un **testigo de compositividad**. Pasar la ronda solo indica que esta base no detectó que el candidato fuera compuesto.

```
Descomponer(N)
    requiere N impar y N ≥ 3
    (s, u) ← (0, N - 1)
    mientras u es par:
        (s, u) ← (s + 1, u / 2)
    devolver (s, u)

RondaMillerRabin(N, a, s, u)
    requiere N ≥ 5 impar, 2 ≤ a ≤ N-2 y N-1 = 2^s·u con u impar
    z ← PotenciaModular(a, u, N)
    si z = 1 o z = N-1:
        devolver verdadero
    para j desde 1 hasta s-1:
        z ← z·z mod N
        si z = N-1:
            devolver verdadero
        si z = 1:
            devolver falso
    devolver falso
```

```
    def descomponer_menos_uno(n):
        """Para n impar >= 3 obtiene n-1 = 2^s*u, con u impar."""
        if n < 3 or n % 2 == 0:
            raise ValueError("Se requiere un entero impar >= 3.")
        s, u = 0, n - 1
        while u % 2 == 0:
            s += 1
            u //= 2
        return s, u

    def ronda_miller_rabin(n, base, s, u):
        """Una ronda; supone n>=5 impar, 2<=base<=n-2 y n-1=2^s*u."""
        z = potencia_modular(base, u, n)
        if z == 1 or z == n - 1:
            return True
        for _ in range(1, s):
            z = z * z % n
            if z == n - 1:
                return True
            if z == 1:
```

```
            return False
    return False
```

3.3. Filtro y varias rondas

Primero descartamos los candidatos divisibles por primos pequeños. Los restantes se someten a $t$$t$ rondas con bases uniformes e independientes.

```
EsProbablementePrimo(N, t)
    requiere t ≥ 1
    si N < 2: devolver falso
    para cada primo r de la lista fija de primos pequeños:
        si N = r: devolver verdadero
        si N mod r = 0: devolver falso
    (s, u) ← Descomponer(N)
    repetir t veces:
        a ←$ {2, ..., N-2}
        si RondaMillerRabin(N, a, s, u) = falso:
            devolver falso
    devolver verdadero
```

Para $N\ge2$$N\ge2$, `False` significa que el número es compuesto. `True` significa que pasó las comprobaciones: es un **primo probable**, salvo cuando coincide con uno de los primos conocidos del filtro.

Para cualquier compuesto impar fijo, como máximo una cuarta parte de las bases posibles permite superar una ronda. Con $t$$t$ elecciones independientes,

$$ \Pr[\text{aceptar este compuesto en las }t\text{ rondas}] \le4^{-t}. $$$$ \Pr[\text{aceptar este compuesto en las }t\text{ rondas}] \le4^{-t}. $$

Con 64 rondas, $4^{-64}=2^{-128}$$4^{-64}=2^{-128}$. Esta probabilidad se refiere a las bases aleatorias para un **compuesto fijo**; no es directamente la probabilidad de que un candidato aceptado sea compuesto. Esta última depende también de cómo se generó. Repetir siempre la misma base no reduce el error.

El ejemplo $2047=23\cdot89$$2047=23\cdot89$ pasa la ronda con base 2 y falla con base 3. Se llama directamente a la ronda porque el filtro completo detectaría antes el factor 23. El segundo ejemplo tiene un factor 3 y se descarta sin ejecutar rondas.

```
    PRIMOS_FILTRO = (
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
        41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97,
    )

    def es_probablemente_primo(n, rondas=64):
        """Miller–Rabin con bases aleatorias; True significa primo probable."""
        if rondas < 1:
            raise ValueError("Se requiere al menos una ronda.")
        if n < 2:
            return False
        for primo in PRIMOS_FILTRO:
            if n == primo:
                return True
            if n % primo == 0:
                return False
        s, u = descomponer_menos_uno(n)
        for _ in range(rondas):
            base = 2 + secrets.randbelow(n - 3)
            if not ronda_miller_rabin(n, base, s, u):
                return False
        return True
```

```
    # 2047 = 23 * 89: una sola base no certifica primalidad.
    s_2047, u_2047 = descomponer_menos_uno(2047)
    print("2047 pasa la ronda con base 2:", ronda_miller_rabin(2047, 2, s_2047, u_2047))
    print("2047 pasa la ronda con base 3:", ronda_miller_rabin(2047, 3, s_2047, u_2047))

    # Este entero grande es compuesto porque tiene el factor 3.
    compuesto_grande = 3 * (2**2046 + 1)
```

```
print("El filtro acepta el compuesto grande:", es_probablemente_primo(compuesto_grande))
```

```
2047 pasa la ronda con base 2: True
2047 pasa la ronda con base 3: False
El filtro acepta el compuesto grande: False
```

3.4. Generación de primos compatibles con RSA

Buscamos un primo probable de $\ell$$\ell$ bits que satisfaga $\gcd(e,p-1)=1$$\gcd(e,p-1)=1$. Sumar $2^{\ell-1}$$2^{\ell-1}$ a $\ell-1$$\ell-1$ bits aleatorios fija el bit superior. Si el candidato es par, sumamos uno; así siempre probamos un impar de exactamente $\ell$$\ell$ bits.

Cada candidato debe superar la comprobación de coprimalidad y Miller–Rabin. Si falla alguna, se genera uno nuevo.

Esta implementación admite $\ell\ge18$$\ell\ge18$ y exponentes impares con $3\le e<2^{\ell-1}$$3\le e<2^{\ell-1}$. Dieciocho es el mínimo número de bits que permite usar $e=65537$$e=65537$ bajo esta condición. Aplicamos ese mínimo también a otros exponentes para excluir las configuraciones diminutas que pueden impedir obtener dos primos compatibles distintos. Por ejemplo, con tres bits y $e=3$$e=3$, solo el primo 5 cumple la coprimalidad exigida. Los experimentos del cuaderno conservan 2048 bits por primo.

```
GenerarPrimoRSA(ℓ, e, t)
    requiere ℓ ≥ 18, e impar, 3 ≤ e < 2^(ℓ-1) y t ≥ 1
    repetir:
        p ← entero de ℓ-1 bits aleatorios + 2^(ℓ-1)
        si p es par: p ← p + 1
        si MCD(e, p-1) ≠ 1: continuar
        si EsProbablementePrimo(p, t): devolver p
```

```
    def generar_primo_rsa(bits, e, rondas=64):
        """Busca un primo probable de bits bits con MCD(e, p-1) = 1."""
        if (bits < 18 or e % 2 == 0
                or not 3 <= e < 2**(bits - 1) or rondas < 1):
            raise ValueError(
                "Se requieren bits >= 18, e impar con 3 <= e < 2**(bits-1) "
                "y rondas >= 1."
            )

        while True:
            candidato = secrets.randbits(bits - 1) + 2**(bits - 1)
            if candidato % 2 == 0:
                candidato += 1
            if mcd(e, candidato - 1) != 1:
                continue
            if es_probablemente_primo(candidato, rondas):
                return candidato
```

El número de intentos varía entre ejecuciones. Al obtener dos factores distintos con $\gcd(e,p-1)=\gcd(e,q-1)=1$, también se cumple $\gcd(e,(p-1)(q-1))=1$. Esto permitirá calcular el exponente privado.

3.5. Generación del par de claves RSA

Reunimos los pasos anteriores: generar $p\ne q$$p\ne q$, formar $n$$n$ y $\varphi(n)$$\varphi(n)$, y calcular $d=e^{-1}\bmod\varphi(n)$$d=e^{-1}\bmod\varphi(n)$. Las demostraciones de RSA suponen factores primos; nuestra implementación los obtiene mediante el test probabilístico ya descrito.

```
GenerarClavesRSA(ℓ, e, t)
    requiere ℓ ≥ 18, e impar, 3 ≤ e < 2^(ℓ-1) y t ≥ 1
    p ← GenerarPrimoRSA(ℓ, e, t)
    q ← GenerarPrimoRSA(ℓ, e, t)
    mientras q = p: q ← GenerarPrimoRSA(ℓ, e, t)
    n ← p·q
    ϕ ← (p-1)·(q-1)
    d ← InversoModular(e, ϕ)
    pk ← (n, e)
```

```
sk ← (n, e, d, p, q, ϕ)
devolver (pk, sk)
```

En Python, las claves son diccionarios. La pública contiene `n,e`; la privada añade `d,p,q,phi`. La inversión directa usa `n,d` y CRT aprovecha los factores. El experimento usa los parámetros de la sección 0: 2048 bits por primo y $e=65537$$e=65537$.

```
def generar_claves_rsa(bits_primo=2048, e=65537, rondas=64):
    """Genera (pk, sk) con dos factores distintos de bits_primo bits."""
    if (bits_primo < 18 or e % 2 == 0
            or not 3 <= e < 2**(bits_primo - 1) or rondas < 1):
        raise ValueError(
            "Se requieren bits >= 18, e impar con 3 <= e < 2**(bits-1) "
            "y rondas >= 1."
        )

    p = generar_primo_rsa(bits_primo, e, rondas)
    print("Primo p generado.")
    q = generar_primo_rsa(bits_primo, e, rondas)
    while q == p:
        q = generar_primo_rsa(bits_primo, e, rondas)
    print("Primo q generado.")

    n = p * q
    phi = (p - 1) * (q - 1)
    d = inverso_modular(e, phi)
    pk = {"n": n, "e": e}
    sk = {"n": n, "e": e, "d": d, "p": p, "q": q, "phi": phi}
    return pk, sk
```

Generación de la clave principal. La búsqueda de primos puede tardar unos minutos. La siguiente celda muestra los factores, el módulo y el tiempo empleado; además comprueba sus tamaños, que $p\ne q$$p\ne q$ y que

$$ ed\bmod\varphi(n)=1. $$$$ ed\bmod\varphi(n)=1. $$

Si vuelve a generar esta clave, ejecute también las celdas posteriores para que todos los experimentos utilicen sus nuevos componentes.

```
inicio_generacion = perf_counter()
pk_rsa, sk_rsa = generar_claves_rsa(BITS_PRIMO, EXPONENTE_PUBLICO, RONDAS_MR)
tiempo_generacion = perf_counter() - inicio_generacion

mostrar_entero("p", sk_rsa["p"])
mostrar_entero("q", sk_rsa["q"])
mostrar_entero("d", sk_rsa["d"])



mostrar_entero("n", pk_rsa["n"])
mostrar_entero("e", pk_rsa["e"])

print("Bits del módulo n:", pk_rsa["n"].bit_length())
print("Los factores son distintos:", sk_rsa["p"] != sk_rsa["q"])
print("Bits del exponente privado d:", sk_rsa["d"].bit_length())
print("(e*d) mod phi(n) =", pk_rsa["e"] * sk_rsa["d"] % sk_rsa["phi"])
print(f"Tiempo de generación: {tiempo_generacion:.2f} s")

assert sk_rsa["p"] != sk_rsa["q"]
assert sk_rsa["p"].bit_length() == sk_rsa["q"].bit_length() == BITS_PRIMO
assert pk_rsa["e"] * sk_rsa["d"] % sk_rsa["phi"] == 1
```

```
Primo p generado.
Primo q generado.
p : 2048 bits
21814833867320483984725697960837723573673013210211007295475365834001557111243267219306740008698610213168393303101695824143853939093891138394420371756324963665193691793664896582635206432777444662610950680919547499893334832128817430877785542928017519291876311974430139466358824629089005019381237476941500368684002536487871757717456001233214831631589607875725252236543553487744901854823144435647290940207439156582762979822876306249869391467693674945000683176811162736065206336326643113144612778611750411174525172874259466690623414124617743033511892801559586506656599854237266479003578212772372646827474246375185822185319
q : 2048 bits
32139431072622559752124435531985531746933151732852863167595280571470980762677998165860469981557714241002103370851177413966206925628354003747359845713892081388364330151063450083
```

```
    838722302544239457316456243967169578242755537492339713963933077369816418006409131550515
    098467426664221445312142914942366357915323284735685723231713091127951892054787665406180
    158427219410485517233333482828914664424289270882074948154895511083132301274264097600134
    705887093129234504172301873115086599172173258566779055979046254766482426546301407710651
    060101844738696381920732078970947068942700419363224133194567772844882719323055950527491
    9
    d : 4094 bits
    174955472156688760159776761375911959588905266411522130525241233316555537811676725619419
    454604202224662570699080519350845696410062288166289958219502236999426617296659772963206
    624753851570390627367431704568689524094588039054139600363980845231432405750488613959009
    363155114688721111897611439290129531958061285016612555316190493840819789938257434153122
    496960888098888570732488892641195107332682944430105615039398037928059614866245886432633
    412892724115289169384887850524300080107279251747792463583371422971070892451890562401966
    749327977004426348401083304408853594704732925475674375192426375585359772380309200922687
    108074448493056798722152145319181385524542798155511161719092277982776236937604939697299
    501575084330553119537950491813535152721515905927197406127601009861114274126833510580251
    527262899905153877663477973656519952516112246495233740190729882126263983728692924126557
    938140969551424340154126096507514649136609785523862052145096672618865304102009824842110
    173253561677306161369214093594464591191247551873219576470948548043253710109688457809084
    168154093660067583669133777114414564104593689426253348292505962078739638358946585684835
    207306228440516887901507632771305066153653393286243652401731606617700298234639965439388
    1
    n : 4096 bits
    701116349439458925925846252310941793786112537899714190304068405764161690202021375010388
    178206897468369261092432432230425241875153001073385409797695860721011508974635657373708
    223706320800335731061475579204977702127125860064274611046484814353209341792208162714420
    818949293833601290842225810001052900570836641563760366745577619838926658504560197021717
    986017226570677525993342580226611455867802502697556053126882060149886448544035505633819
    386067656863379374708168953149752620153525762614349681170564568133488631443044624442808
    864357810256762235242863918371226491143700729907011833617833397134506750794198612013583
    912471298713938412044646935257399146279435830836883966070363466754811366276195345116311
    804561055080335711472376416220940350828430639993652627660036576753291352099753955707230
    444041776234247999127074037490810763353679234642444513734947450242126630316956604615945
    804213760105386360482161838490541398738990260432658350165751457703725160324797689677829
    205852692034243696976038420170127598444001293061996519626829323625204596168460554872985
    888056525877868836995806160620179980800462276928099729026976835402918404101770535674820
    706971502738555282261830823808991737961127966577246929819206551807946689344900662341249
    91
    e : 17 bits
    65537
    Bits del módulo n: 4096
    Los factores son distintos: True
    Bits del exponente privado d: 4094
    (e*d) mod phi(n) = 1
```

Los factores deben tener 2048 bits y el módulo, 4095 o 4096. La igualdad entre los exponentes comprueba el cálculo del inverso. Los factores se muestran para observar la construcción: Alice los conserva como información privada y entrega a Bob únicamente $(n,e)$$(n,e)$.

4. RSA en funcionamiento

4.1. Evaluación e inversión

Con la clave generada, implementamos $F_{\mathrm{RSA}}$$F_{\mathrm{RSA}}$ e $I_{\mathrm{RSA}}$$I_{\mathrm{RSA}}$ mediante la exponenciación modular. Ambas operaciones requieren un entero en $[0,n)$$[0,n)$; una entrada fuera de ese intervalo se rechaza.

```
EvaluarRSA(pk, x)
    (n, e) ← componentes de pk
    si x < 0 o x ≥ n: rechazar
    devolver PotenciaModular(x, e, n)

InvertirRSA(sk, y)
    (n, d) ← componentes correspondientes de sk
    si y < 0 o y ≥ n: rechazar
    devolver PotenciaModular(y, d, n)
```

El experimento elige una entrada uniforme, calcula su imagen pública y comprueba que la inversión privada recupera exactamente esa entrada. Después justificaremos que esta propiedad vale para todo el dominio.

```
    def validar_elemento_rsa(valor, modulo):
        if not isinstance(valor, int) or not 0 <= valor < modulo:
            raise ValueError("La entrada RSA debe ser un entero en [0, n).")

    def evaluar_rsa(pk, x):
```

```
    """F_RSA(pk,x): evaluación pública de la permutación RSA."""
    validar_elemento_rsa(x, pk["n"])
    return potencia_modular(x, pk["e"], pk["n"])

def invertir_rsa(sk, y):
    """I_RSA(sk,y): inversión directa de la permutación RSA."""
    validar_elemento_rsa(y, sk["n"])
    return potencia_modular(y, sk["d"], sk["n"])
```

```
x_rsa = secrets.randbelow(pk_rsa["n"])
y_rsa = evaluar_rsa(pk_rsa, x_rsa)
x_recuperado = invertir_rsa(sk_rsa, y_rsa)
print("x_rsa:", x_rsa)
print("Bits de la entrada x:", x_rsa.bit_length())
print("y_rsa:", y_rsa)
print("Bits de la salida y:", y_rsa.bit_length())
print("x_recuperado:", x_recuperado)
print("La inversión recupera x:", x_recuperado == x_rsa)
assert x_recuperado == x_rsa
```

```
x_rsa: 340133435739577462343249302119236056019848201782158254467504120085309446142013814963570946459795370360619441696063725455
Bits de la entrada x: 4095
y_rsa: 281246539865656326762734360327599061015994618025593650913478957164688364593240468916421252619096046062993465677650058633
Bits de la salida y: 4095
x_recuperado: 340133435739577462343249302119236056019848201782158254467504120085309446142013814963570946459795370360619441696063
La inversión recupera x: True
```

4.2. Correctitud de RSA

Como $ed\equiv1\pmod{\varphi(n)}$$ed\equiv1\pmod{\varphi(n)}$, existe un entero $h\ge0$$h\ge0$ tal que

$$ ed=1+h(p-1)(q-1). $$$$ ed=1+h(p-1)(q-1). $$

Si $x\not\equiv0\pmod p$$x\not\equiv0\pmod p$, el pequeño teorema de Fermat da

$$ x^{ed}=x\left(x^{p-1}\right)^{h(q-1)}\equiv x\pmod p. $$$$ x^{ed}=x\left(x^{p-1}\right)^{h(q-1)}\equiv x\pmod p. $$

Si $x\equiv0\pmod p$$x\equiv0\pmod p$, la igualdad también se cumple porque $ed>0$$ed>0$. El mismo argumento módulo $q$$q$ demuestra

$$ x^{ed}\equiv x\pmod p,\qquad x^{ed}\equiv x\pmod q. $$$$ x^{ed}\equiv x\pmod p,\qquad x^{ed}\equiv x\pmod q. $$

Por tanto, $p$$p$ y $q$$q$ dividen a $x^{ed}-x$$x^{ed}-x$. Como son coprimos, su producto también lo divide, de modo que $x^{ed}\equiv x\pmod n$$x^{ed}\equiv x\pmod n$. Al elegir el representante en $[0,n)$$[0,n)$, obtenemos

$$ I_{\mathrm{RSA}}(sk,F_{\mathrm{RSA}}(pk,x))=x. $$$$ I_{\mathrm{RSA}}(sk,F_{\mathrm{RSA}}(pk,x))=x. $$

Esto vale para **toda entrada de $\mathbb Z_n$$\mathbb Z_n$**, incluso si comparte un factor con $n$$n$. Las pruebas incluyen $0,1,p,q,n-1$$0,1,p,q,n-1$ y una entrada aleatoria.

```
casos_rsa = {
    "cero": 0,
    "uno": 1,
    "múltiplo de p": sk_rsa["p"],
    "múltiplo de q": sk_rsa["q"],
    "n - 1": pk_rsa["n"] - 1,
    "entrada aleatoria": x_rsa,
}
for nombre, x_caso in casos_rsa.items():
    y_caso = evaluar_rsa(pk_rsa, x_caso)
    assert invertir_rsa(sk_rsa, y_caso) == x_caso
    print(f"{nombre}: recuperación exacta")

try:
    evaluar_rsa(pk_rsa, pk_rsa["n"]+1)
except ValueError as error:
    print("Entrada fuera del dominio:", error)
```

```
cero: recuperación exacta
uno: recuperación exacta
múltiplo de p: recuperación exacta
múltiplo de q: recuperación exacta
```

```
    n - 1: recuperación exacta
    entrada aleatoria: recuperación exacta
    Entrada fuera del dominio: La entrada RSA debe ser un entero en [0, n).
```

Las entradas $p$$p$ y $q$$q$ comprueban los casos no invertibles; $n$$n$ se rechaza por estar fuera del dominio. Los factores solo se usan para preparar estas pruebas.

Ya sabemos que RSA recupera la entrada. Ahora usaremos sus residuos módulo $p$$p$ y módulo $q$$q$ para calcular la inversión con menos trabajo.

5. RSA-CRT: teorema chino del resto

5.1. Dos operaciones más pequeñas

Para obtener $x=y^d\bmod n$$x=y^d\bmod n$, primero calcularemos los residuos de $y^d$$y^d$ módulo $p$$p$ y módulo $q$$q$. Estos cálculos pueden hacerse con exponentes reducidos:

$$ d_p=d\bmod(p-1),\qquad d_q=d\bmod(q-1). $$$$ d_p=d\bmod(p-1),\qquad d_q=d\bmod(q-1). $$

En efecto, podemos escribir $d=k_p(p-1)+d_p$$d=k_p(p-1)+d_p$ para algún entero $k_p\ge0$$k_p\ge0$. Si $y\not\equiv0\pmod p$$y\not\equiv0\pmod p$, el pequeño teorema de Fermat da

$$ y^d =y^{d_p}\left(y^{p-1}\right)^{k_p} \equiv y^{d_p}\pmod p. $$$$ y^d =y^{d_p}\left(y^{p-1}\right)^{k_p} \equiv y^{d_p}\pmod p. $$

Si $y\equiv0\pmod p$$y\equiv0\pmod p$, ambas potencias valen cero, pues $d$$d$ y $d_p$$d_p$ son positivos. El mismo argumento se aplica módulo $q$$q$. Por tanto,

$$ x_p=y^{d_p}\bmod p=y^d\bmod p, $$$$ x_p=y^{d_p}\bmod p=y^d\bmod p, $$

$$ x_q=y^{d_q}\bmod q=y^d\bmod q. $$$$ x_q=y^{d_q}\bmod q=y^d\bmod q. $$

Así, $x_p$$x_p$ y $x_q$$x_q$ son precisamente los dos residuos de $y^d$$y^d$ que necesitamos. El teorema chino del resto permite reconstruir el único $x\in[0,pq)$$x\in[0,pq)$ que satisface

$$ x\equiv x_p\equiv y^d\pmod p, \qquad x\equiv x_q\equiv y^d\pmod q. $$$$ x\equiv x_p\equiv y^d\pmod p, \qquad x\equiv x_q\equiv y^d\pmod q. $$

Como $p$$p$ y $q$$q$ son coprimos, estas congruencias implican $x\equiv y^d\pmod{pq}$$x\equiv y^d\pmod{pq}$. En consecuencia,

$$ x=y^d\bmod n. $$$$ x=y^d\bmod n. $$

Obtenemos la misma salida que con la inversión directa, mediante dos exponenciaciones con módulos de aproximadamente la mitad de longitud que $n$$n$ y exponentes reducidos.

5.2. Recombinar los residuos

Una vez calculados $x_p$$x_p$ y $x_q$$x_q$, reconstruimos $x$$x$ a partir de ambos residuos. Comenzamos por la condición módulo $q$$q$: todo entero que satisface $x\equiv x_q\pmod q$$x\equiv x_q\pmod q$ puede escribirse como

$$ x=x_q+qh $$$$ x=x_q+qh $$

para algún entero $h$$h$. Falta elegir $h$$h$ de modo que también se cumpla $x\equiv x_p\pmod p$$x\equiv x_p\pmod p$. Sustituyendo la expresión anterior, necesitamos

$$ x_q+qh\equiv x_p\pmod p, $$$$ x_q+qh\equiv x_p\pmod p, $$

o, equivalentemente,

$$ qh\equiv x_p-x_q\pmod p. $$$$ qh\equiv x_p-x_q\pmod p. $$

Como $p$$p$ y $q$$q$ son primos distintos, $q$$q$ tiene inverso módulo $p$$p$. Definimos $q_{\mathrm{inv}}=q^{-1}\bmod p$$q_{\mathrm{inv}}=q^{-1}\bmod p$ y multiplicamos la congruencia por ese inverso. Esto permite elegir

$$ h=q_{\mathrm{inv}}(x_p-x_q)\bmod p. $$$$ h=q_{\mathrm{inv}}(x_p-x_q)\bmod p. $$

Con este valor, la recombinación queda

$$ \boxed{x=x_q+qh.} $$$$ \boxed{x=x_q+qh.} $$

Por construcción, $x$$x$ tiene los residuos requeridos módulo $p$$p$ y módulo $q$$q$. Además, como $0\le x_q<q$$0\le x_q<q$ y $0\le h<p$$0\le h<p$,

$$ 0\le x\le(q-1)+q(p-1)=pq-1. $$$$ 0\le x\le(q-1)+q(p-1)=pq-1. $$

Por tanto, $x$$x$ ya pertenece a $[0,n)$$[0,n)$ y es exactamente el resultado $y^d\bmod n$$y^d\bmod n$ que queríamos obtener.

5.3. Preparar, calcular y recombinar

Los valores $d_p,d_q,q_{\mathrm{inv}}$$d_p,d_q,q_{\mathrm{inv}}$ se calculan una sola vez por clave. Cada inversión obtiene los dos residuos y llama a la función de recombinación. Esta separación permitirá estudiar después una falla en uno de los residuos.

```
PrepararClaveCRT(sk)
    d_p ← d mod (p-1)
    d_q ← d mod (q-1)
    q_inv ← InversoModular(q, p)
    devolver sk con los componentes adicionales (d_p, d_q, q_inv)

RecombinarRSACRT(x_p, x_q, sk_crt)
    h ← q_inv·(x_p - x_q) mod p
    devolver x_q + q·h

InvertirRSACRT(sk_crt, y)
    si y < 0 o y ≥ n: rechazar
    x_p ← PotenciaModular(y, d_p, p)
    x_q ← PotenciaModular(y, d_q, q)
    devolver RecombinarRSACRT(x_p, x_q, sk_crt)
```

El experimento aplica CRT a los mismos seis casos del bloque 4 y comprueba que recupera las entradas originales.

```
    def preparar_clave_crt(sk):
        sk_crt = sk.copy()
        sk_crt["dp"] = sk["d"] % (sk["p"] - 1)
        sk_crt["dq"] = sk["d"] % (sk["q"] - 1)
        sk_crt["qinv"] = inverso_modular(sk["q"], sk["p"])
        return sk_crt

    def recombinar_rsa_crt(xp, xq, sk_crt):
        h = sk_crt["qinv"] * (xp - xq) % sk_crt["p"]
        return xq + sk_crt["q"] * h

    def invertir_rsa_crt(sk_crt, y):
        validar_elemento_rsa(y, sk_crt["n"])
        xp = potencia_modular(y, sk_crt["dp"], sk_crt["p"])
        xq = potencia_modular(y, sk_crt["dq"], sk_crt["q"])
        return recombinar_rsa_crt(xp, xq, sk_crt)
```

```
    sk_crt = preparar_clave_crt(sk_rsa)
    for nombre, x_caso in casos_rsa.items():
        y_caso = evaluar_rsa(pk_rsa, x_caso)
        recuperado_crt = invertir_rsa_crt(sk_crt, y_caso)
        print(nombre, ":", recuperado_crt == x_caso)
        assert recuperado_crt == x_caso
```

```
    cero : True
    uno : True
    múltiplo de p : True
    múltiplo de q : True
    n - 1 : True
    entrada aleatoria : True
```

5.4. Comparar los tiempos

Aplicamos ambas inversiones a **la misma clave y entrada**. Repetimos cada operación tres veces y tomamos la mediana; la preparación de la clave CRT queda fuera de la medición.

El cociente `tiempo_directo / tiempo_crt` es la aceleración observada: un valor 3 significa que la inversión directa tardó tres veces lo que tardó CRT. Depende del entorno y de la aritmética utilizada. En cada repetición se comprueba también que ambos resultados coincidan.

```
    tiempos_directos = []
    tiempos_crt = []
```

```
    for repeticion in range(3):
        inicio = perf_counter()
        resultado_directo = invertir_rsa(sk_rsa, y_rsa)
        tiempos_directos.append(perf_counter() - inicio)

        inicio = perf_counter()
        resultado_crt = invertir_rsa_crt(sk_crt, y_rsa)
        tiempos_crt.append(perf_counter() - inicio)

        assert resultado_directo == resultado_crt == x_rsa

    tiempo_directo = median(tiempos_directos)
    tiempo_crt = median(tiempos_crt)
```

```
    print(f"Inversión directa: {1000 * tiempo_directo:.2f} ms")
    print(f"Inversión con CRT: {1000 * tiempo_crt:.2f} ms")
    print(f"Razón medida, directa / CRT: {tiempo_directo / tiempo_crt:.2f}")
```

```
    Inversión directa: 327.22 ms
    Inversión con CRT: 92.03 ms
    Razón medida, directa / CRT: 3.56
```

6. Seguridad y ataques

La correctitud y la rapidez de RSA no bastan para garantizar su seguridad en una aplicación. Estudiaremos cuatro ataques y precisaremos, en cada uno, qué conoce el atacante, qué condición explota y qué recupera.

Algunos ataques obtienen una entrada concreta; otros obtienen un factor y permiten reconstruir la clave privada. Empezaremos por ese último paso, que será común a dos experimentos.

6.1. De un factor a la clave privada

Si se conoce un divisor no trivial $f$$f$ de un módulo RSA válido $n=pq$$n=pq$, el otro factor y el exponente privado se obtienen mediante

$$ q=n/f,\qquad \varphi(n)=(f-1)(q-1),\qquad d=e^{-1}\bmod\varphi(n). $$$$ q=n/f,\qquad \varphi(n)=(f-1)(q-1),\qquad d=e^{-1}\bmod\varphi(n). $$

```
RecuperarClaveDesdeFactor(pk, f)
    requiere que pk corresponda a un módulo RSA válido con dos factores
    si f ≤ 1 o f ≥ n o n mod f ≠ 0: rechazar
    (p, q) ← (f, n / f)
    ϕ ← (p-1)·(q-1)
    d ← InversoModular(e, ϕ)
    devolver (n, e, d, p, q, ϕ)
```

La función recibe un factor ya conocido. Los ataques 6.3 y 6.5 lo proporcionarán usando información pública o una salida defectuosa. La clave reconstruida permitirá invertir cualquier entrada de ese módulo.

```
    def recuperar_clave_desde_factor(pk, factor):
        """Recupera la clave de un módulo RSA válido dado un divisor no trivial."""
        n_rsa, e = pk["n"], pk["e"]
        if not 1 < factor < n_rsa or n_rsa % factor != 0:
            raise ValueError("Se requiere un divisor no trivial del módulo.")
        p, q = factor, n_rsa // factor
        phi = (p - 1) * (q - 1)
        d = inverso_modular(e, phi)
        return {"n": n_rsa, "e": e, "d": d, "p": p, "q": q, "phi": phi}
```

```
    pk_rsa, sk_rsa = generar_claves_rsa(BITS_PRIMO, EXPONENTE_PUBLICO, RONDAS_MR)
```

```
    Primo p generado.
    Primo q generado.
```

```
    f=sk_rsa['p']
    dicc=recuperar_clave_desde_factor(pk_rsa, f)


    print("p",sk_rsa['p'])
    print("q",sk_rsa['q'])
    print("d",sk_rsa['d'])

    print("n:",dicc['n'])
    print("e:",dicc['e'])
    print("d:",dicc['d'])

    print("p:",dicc['p'])
    print("q:",dicc['q'])
    print("phi:", dicc['phi'])
```

```
p 164078424842868610446483803613322036951784529694001064998477872356965096760354143988995868042050659877224450684513576885505063
q 322227973566726979809130430629978468066933271363998414815614797153584879041613341489666689672079209961228429961662776319434999
d 262001674017744933379482276136677748204429842965990939297769881146571825094003619228123678499313483822475242882399660457052528
n: 528706583431380660125354248581132788868236555669001083497796739252482609267657579005251147513917781484544754527321690654119
e: 65537
d: 262001674017744933379482276136677748204429842965990939297769881146571825094003619228123678499313483822475242882399660457052528
p: 164078424842868610446483803613322036951784529694001064998477872356965096760354143988995868042050659877224450684513576885505063
q: 322227973566726979809130430629978468066933271363998414815614797153584879041613341489666689672079209961228429961662776319434999
phi: 528706583431380660125354248581132788868236555669001083497796739252482609267657579005251147513917781484544754527321690654119
```

6.2. PIN cifrado dígito a dígito

El atacante conoce la clave pública y los ocho cifrados de los dígitos de un PIN. Cada dígito pertenece a $\{0,\ldots,9\}$$\{0,\ldots,9\}$ y RSA directo es determinista, por lo que puede construir la tabla

$$ D[F_{\mathrm{RSA}}(pk,j)]=j,\qquad j\in\{0,\ldots,9\}. $$$$ D[F_{\mathrm{RSA}}(pk,j)]=j,\qquad j\in\{0,\ldots,9\}. $$

Como RSA es una permutación, esas diez imágenes son distintas. Cada cifrado identifica su dígito con una consulta a la tabla.

```
RecuperarPIN(pk, C)
    D ← diccionario vacío
    para j desde 0 hasta 9:
        D[EvaluarRSA(pk, j)] ← representación decimal de j
    pin ← cadena vacía
    para cada c en C:
        si c no pertenece a D: rechazar la hipótesis de codificación
        pin ← pin concatenado con D[c]
    devolver pin
```

El ataque utiliza diez evaluaciones públicas y ocho consultas, sin conocer $d,p,q$$d,p,q$. El PIN se representa como cadena para conservar los posibles ceros iniciales.

```
    def recuperar_pin(pk, cifrados):
        """Solo necesita la clave pública y los dígitos cifrados."""
        tabla = {}
        for digito in range(10):
            cifrado = evaluar_rsa(pk, digito)
            tabla[cifrado] = str(digito)

        pin = ""
        for cifrado in cifrados:
            if cifrado not in tabla:
                raise ValueError("El cifrado no corresponde a un dígito.")
            pin += tabla[cifrado]
        return pin
```

```
    # Guardarlo como texto conserva los posibles ceros al comienzo.
    pin_original = ""
    for posicion in range(8):
        pin_original += str(secrets.randbelow(10))

    print("PIN original:  ", pin_original)
```

```
    cifrados_pin = []
    for digito in pin_original:
        cifrados_pin.append(evaluar_rsa(pk_rsa, int(digito)))
        print(f"Cifrado {digito}:", cifrados_pin[-1])


    #print("PIN original:  ", cifrados_pin)
```

```
    PIN original:   11068623
    Cifrado 1: 1
    Cifrado 1: 1
    Cifrado 0: 0
    Cifrado 6: 263665606302206215273369857459374446152053620704844143263362562607467504126661467226986209857640149516823203720150821
    Cifrado 8: 278150126642407571843504725391949325251120754950411456838792218741670692630962351853809048344116692780621041898670545
    Cifrado 6: 263665606302206215273369857459374446152053620704844143263362562607467504126661467226986209857640149516823203720150821
    Cifrado 2: 313890164994186481547463524354617395838292542682234692201723402766359381711345282094508414421846442629915002373190891
    Cifrado 3: 186013786941019003442536030898949141416625702055114725400898646571059552281806441400798404488856684308794561407766883
```

```
    pin_recuperado = recuperar_pin(pk_rsa, cifrados_pin)
    print("PIN recuperado:", pin_recuperado)
    print("Evaluaciones públicas para construir la tabla: 10")
    assert pin_recuperado == pin_original
```

```
    PIN recuperado: 11068623
    Evaluaciones públicas para construir la tabla: 10
```

El experimento recupera el PIN, pero no la clave privada. Aumentar el tamaño del módulo no amplía las diez posibilidades de cada posición. Esto no contradice la unidireccionalidad: en aquel juego, la entrada se elige uniformemente en todo el dominio.

Cifrar el PIN completo como un entero elevaría la búsqueda a $10^8$$10^8$ candidatos, que seguirían siendo comprobables públicamente. OAEP abordará este problema incorporando aleatoriedad al cifrado.

6.3. Primos compartidos entre dos módulos

Ahora la debilidad está en la generación de claves. Supongamos que dos módulos públicos distintos comparten un primo:

$$ n_1=pq_1,\qquad n_2=pq_2, $$$$ n_1=pq_1,\qquad n_2=pq_2, $$

con $p,q_1,q_2$$p,q_1,q_2$ distintos. El atacante obtiene directamente

$$ f=\gcd(n_1,n_2)=p. $$$$ f=\gcd(n_1,n_2)=p. $$

Al dividir ambos módulos por $f$$f$, obtiene los demás factores. La función de 6.1 permite reconstruir **ambas claves privadas**. No necesita observar ningún mensaje cifrado.

El procedimiento devuelve el MCD solo si es un divisor no trivial de ambos módulos:

```
FactorCompartido(pk_1, pk_2)
    n_1 ← módulo de pk_1
    n_2 ← módulo de pk_2
    f ← MCD(n_1, n_2)
    si 1 < f < n_1 y f < n_2:
        devolver f
    devolver ⊥
```

`None` indica que no se encontró un factor mediante esta comparación. Si se compara un módulo consigo mismo, el MCD es el módulo completo y no se obtiene una factorización.

```
    def factor_compartido(pk_1, pk_2):
        """Atacante: busca un factor no trivial usando solo dos claves públicas."""
        n_1, n_2 = pk_1["n"], pk_2["n"]
        factor = mcd(n_1, n_2)
        if 1 < factor < n_1 and factor < n_2:
            return factor
        return None
```

Experimento. Reutilizamos $n_1=pq_1$$n_1=pq_1$ y generamos otro primo $q_2$$q_2$, distinto de los anteriores, para construir deliberadamente $n_2=pq_2$$n_2=pq_2$. La generación puede tardar.

Después, el ataque recibe solo las claves públicas. Con las claves privadas que reconstruye, invierte un reto de cada módulo; las comparaciones con los originales comprueban el resultado.

```
# Reutilizamos p deliberadamente y buscamos un tercer primo distinto.
pk_rsa, sk_rsa = generar_claves_rsa(BITS_PRIMO, EXPONENTE_PUBLICO, RONDAS_MR)

q_adicional = generar_primo_rsa(BITS_PRIMO, EXPONENTE_PUBLICO, RONDAS_MR)

while q_adicional == sk_rsa["p"] or q_adicional == sk_rsa["q"]:
    q_adicional = generar_primo_rsa(BITS_PRIMO, EXPONENTE_PUBLICO, RONDAS_MR)

pk_factor_1 = pk_rsa.copy()

pk_factor_2 = {"n": sk_rsa["p"] * q_adicional, "e": EXPONENTE_PUBLICO}

print("Bits de los módulos:", pk_factor_1["n"].bit_length(), pk_factor_2["n"].bit_length())
```

```
Primo p generado.
Primo q generado.
Bits de los módulos: 4095 4095
```

```
print(pk_factor_1 ) ## n1=p*q
print(pk_factor_2)  ## n2=p*q_adicional
```

```
{'n': 339743320236930667612326256162994878298251503705444787062863577939913921328005416134130697517783726033870291625813167166090
{'n': 386568108516583149770809691103952394471866286679624853118891104009700776450346483532707162378151020241043057619284399906799
```

```
print("p", sk_rsa["p"])
print("q", sk_rsa["q"])
print("q_adicional", q_adicional)

p= factor_compartido(pk_factor_1, pk_factor_2)
print(p)

print(pk_factor_1["n"]//p)

print(pk_factor_2["n"]//p)
```

```
p 16265648575769537595579445651556559279252432479963814314566668738492376869801254924857329426045711375476652468952667168078610512
q 20887167127355523063898179053653529128468306741756465841499948829146272307761010576776568586187450281783470484601115901489927106
q_adicional 23765920351460340127680047767251252598516574423440776620657804933867205912913686337739091755917712298185697790966082960
16265648575769537595579445651556559279252432479963814314566668738492376869801254924857329426045711375476652468952667168078610512
20887167127355523063898179053653529128468306741756465841499948829146272307761010576776568586187450281783470484601115901489927106
23765920351460340127680047767251252598516574423440776620657804933867205912913686337739091755917712298185697790966082960149754797
```

```
Empieza a programar o a crear código con IA.
```

Miller–Rabin no detecta si otra clave ya utilizó el mismo factor. Esta debilidad se ha observado en sistemas reales, como documenta Mining Your Ps and Qs.

Recuperar ambas claves permite descifrar también mensajes protegidos con OAEP. El tiempo mostrado corresponde al MCD y a la reconstrucción de las claves; excluye la generación del primo y la inversión de los retos.

6.4. Módulo común y exponentes coprimos

El atacante conoce $(n,e_1),(n,e_2)$$(n,e_1),(n,e_2)$ y dos imágenes de la **misma entrada** $x$$x$, con $\gcd(x,n)=1$$\gcd(x,n)=1$ y $\gcd(e_1,e_2)=1$$\gcd(e_1,e_2)=1$:

$$ y_1=x^{e_1}\bmod n,\qquad y_2=x^{e_2}\bmod n. $$$$ y_1=x^{e_1}\bmod n,\qquad y_2=x^{e_2}\bmod n. $$

Euclides extendido obtiene $u,v$$u,v$ tales que $ue_1+ve_2=1$$ue_1+ve_2=1$. Entonces

$$ y_1^u y_2^v\equiv x^{ue_1+ve_2}=x\pmod n. $$$$ y_1^u y_2^v\equiv x^{ue_1+ve_2}=x\pmod n. $$

Como ambos exponentes públicos son mayores que uno, uno de los coeficientes es negativo. Se calcula $a^{-t}$$a^{-t}$ como $(a^{-1})^t$$(a^{-1})^t$. La hipótesis $\gcd(x,n)=1$$\gcd(x,n)=1$ garantiza que las imágenes tengan los inversos necesarios.

El auxiliar maneja los exponentes negativos mediante un inverso modular. El ataque comprueba sus hipótesis y verifica que la entrada recuperada produzca las dos imágenes observadas.

```
PotenciaConExponenteEntero(a, b, N)
    requiere N ≥ 2 y b entero
    si b < 0:
        a ← InversoModular(a, N)
        b ← -b
    devolver PotenciaModular(a, b, N)

RecuperarPorModuloComun(pk_1, y_1, pk_2, y_2)
    requiere que pk_1 y pk_2 sean claves RSA válidas
    si sus módulos son distintos: rechazar
    n ← módulo común
    validar que y_1, y_2 pertenecen a [0, n)
    (g, u, v) ← EuclidesExtendido(e_1, e_2)
    si g ≠ 1: rechazar
    si MCD(y_1, n) ≠ 1 o MCD(y_2, n) ≠ 1: rechazar
    a ← PotenciaConExponenteEntero(y_1, u, n)
    b ← PotenciaConExponenteEntero(y_2, v, n)
    x ← (a·b) mod n
    si EvaluarRSA(pk_1, x) ≠ y_1 o EvaluarRSA(pk_2, x) ≠ y_2:
        rechazar: los criptogramas no corresponden a la misma entrada
    devolver x
```

```
    def potencia_con_exponente_entero(base, exponente, modulo):
        """Para exponentes negativos, obtiene primero el inverso de la base."""
        if exponente < 0:
            base = inverso_modular(base, modulo)
            exponente = -exponente
        return potencia_modular(base, exponente, modulo)

    def recuperar_por_modulo_comun(pk_1, y_1, pk_2, y_2):
        """Atacante: recibe solo claves públicas y dos imágenes RSA relacionadas."""
        if pk_1["n"] != pk_2["n"]:
            raise ValueError("Este ataque requiere un módulo común.")
        modulo = pk_1["n"]
        validar_elemento_rsa(y_1, modulo)
        validar_elemento_rsa(y_2, modulo)
        divisor, u, v = euclides_extendido(pk_1["e"], pk_2["e"])
        if divisor != 1:
            raise ValueError("Los exponentes públicos deben ser coprimos.")
        if mcd(y_1, modulo) != 1 or mcd(y_2, modulo) != 1:
            raise ValueError("Esta variante requiere criptogramas invertibles.")

        termino_1 = potencia_con_exponente_entero(y_1, u, modulo)
        termino_2 = potencia_con_exponente_entero(y_2, v, modulo)
        recuperado = termino_1 * termino_2 % modulo
        if (evaluar_rsa(pk_1, recuperado) != y_1
                or evaluar_rsa(pk_2, recuperado) != y_2):
            raise ValueError("Los criptogramas no corresponden a la misma entrada.")
        return recuperado
```

Experimento. Conservamos $n,e_1$$n,e_1$ y elegimos $e_2$$e_2$ coprimo con $e_1$$e_1$ y con $\varphi(n)$$\varphi(n)$, de modo que sea otro exponente RSA válido. Luego generamos las dos imágenes de una entrada invertible. Solo esta preparación consulta $\varphi(n)$$\varphi(n)$.

El ataque muestra la identidad de Bézout y recupera la entrada. Después se cambia la segunda entrada para comprobar que la verificación detecta el incumplimiento de la hipótesis.

```
# Buscamos un segundo exponente coprimo con e_1 y con phi(n).
e_comun_1 = pk_rsa["e"]
e_comun_2 = e_comun_1 + 2
while mcd(e_comun_1, e_comun_2) != 1 or mcd(e_comun_2, sk_rsa["phi"]) != 1:
    e_comun_2 += 2

pk_comun_1 = pk_rsa.copy()
pk_comun_2 = {"n": pk_rsa["n"], "e": e_comun_2}

# Elegimos una entrada invertible módulo n.
x_modulo_comun = secrets.randbelow(pk_rsa["n"])
while mcd(x_modulo_comun, pk_rsa["n"]) != 1:
    x_modulo_comun = secrets.randbelow(pk_rsa["n"])

y_comun_1 = evaluar_rsa(pk_comun_1, x_modulo_comun)
y_comun_2 = evaluar_rsa(pk_comun_2, x_modulo_comun)
print("Exponentes públicos:", e_comun_1, "y", e_comun_2)
```

```
Exponentes públicos: 65537 y 65539
```

```
inicio_modulo_comun = perf_counter()
x_modulo_comun_recuperado = recuperar_por_modulo_comun(
    pk_comun_1, y_comun_1, pk_comun_2, y_comun_2
)
tiempo_modulo_comun = perf_counter() - inicio_modulo_comun

g_comun, u_comun, v_comun = euclides_extendido(e_comun_1, e_comun_2)
assert g_comun == 1
assert u_comun * e_comun_1 + v_comun * e_comun_2 == 1
assert u_comun * v_comun < 0
assert x_modulo_comun_recuperado == x_modulo_comun
print(f"Bézout: ({u_comun})·{e_comun_1} + ({v_comun})·{e_comun_2} = 1")
print("Coeficiente que requiere un inverso:", "u" if u_comun < 0 else "v")
print("Entrada recuperada exactamente:", x_modulo_comun_recuperado == x_modulo_comun)
print(f"Tiempo del ataque, incluidas las verificaciones: {1000 * tiempo_modulo_comun:.2f} ms")

# Cambiar solo el mensaje del segundo cifrado rompe la hipótesis del ataque.
# Como x es invertible y n es impar, 2*x mod n es invertible y distinto de x.
x_comun_distinto = 2 * x_modulo_comun % pk_comun_2["n"]
y_comun_distinto = evaluar_rsa(pk_comun_2, x_comun_distinto)
try:
    recuperar_por_modulo_comun(pk_comun_1, y_comun_1, pk_comun_2, y_comun_distinto)
except ValueError as error:
    print("Dos entradas distintas:", error)
```

```
Bézout: (32769)·65537 + (-32768)·65539 = 1
Coeficiente que requiere un inverso: v
Entrada recuperada exactamente: True
Tiempo del ataque, incluidas las verificaciones: 18.33 ms
Dos entradas distintas: Los criptogramas no corresponden a la misma entrada.
```

Se recupera la entrada común sin calcular la clave privada. Si las entradas originales son distintas, ninguna entrada puede producir ambas imágenes: cada evaluación RSA es una permutación.

La igualdad exigida es la del **entero de entrada a RSA**. Dos cifrados OAEP del mismo texto, con semillas distintas, producen enteros distintos; aun así, compartir un módulo entre usuarios sigue siendo una mala práctica.

Esta variante rechaza imágenes no invertibles. Si alguna satisface $1<\gcd(y_i,n)<n$$1<\gcd(y_i,n)<n$, ese MCD ya revela un factor y permite aplicar 6.1.

6.5. Una falla en RSA-CRT

La inversión privada recibe una entrada $y$$y$ y debe devolver $x=y^d\bmod n$$x=y^d\bmod n$, donde $n=pq$$n=pq$ y $p,q$$p,q$ son primos distintos. Recordemos cómo se obtiene esa salida mediante CRT. Primero se calculan los exponentes reducidos

$$ d_p=d\bmod(p-1),\qquad d_q=d\bmod(q-1), $$$$ d_p=d\bmod(p-1),\qquad d_q=d\bmod(q-1), $$

y después los dos residuos de la salida correcta:

$$ x_p=y^{d_p}\bmod p,\qquad x_q=y^{d_q}\bmod q. $$$$ x_p=y^{d_p}\bmod p,\qquad x_q=y^{d_q}\bmod q. $$

Para recombinarlos, buscamos un entero de la forma $x=x_q+qh$$x=x_q+qh$. Esta forma garantiza $x\equiv x_q\pmod q$$x\equiv x_q\pmod q$. Para que además se cumpla $x\equiv x_p\pmod p$$x\equiv x_p\pmod p$, necesitamos

$$ x_q+qh\equiv x_p\pmod p \quad\Longleftrightarrow\quad qh\equiv x_p-x_q\pmod p. $$$$ x_q+qh\equiv x_p\pmod p \quad\Longleftrightarrow\quad qh\equiv x_p-x_q\pmod p. $$

Como $p$$p$ y $q$$q$ son primos distintos, existe $q_{\mathrm{inv}}=q^{-1}\bmod p$$q_{\mathrm{inv}}=q^{-1}\bmod p$. Por tanto, elegimos

$$ h=q_{\mathrm{inv}}(x_p-x_q)\bmod p, \qquad \boxed{x=x_q+qh.} $$$$ h=q_{\mathrm{inv}}(x_p-x_q)\bmod p, \qquad \boxed{x=x_q+qh.} $$

Al reducir esta expresión módulo $p$$p$, obtenemos

$$ x\equiv x_q+q\,q_{\mathrm{inv}}(x_p-x_q) \equiv x_q+(x_p-x_q)=x_p\pmod p. $$$$ x\equiv x_q+q\,q_{\mathrm{inv}}(x_p-x_q) \equiv x_q+(x_p-x_q)=x_p\pmod p. $$

Así, $x$$x$ tiene los dos residuos requeridos. Además, $0\le x_q<q$$0\le x_q<q$ y $0\le h<p$$0\le h<p$ implican $0\le x<pq$$0\le x<pq$, de modo que la recombinación devuelve precisamente $y^d\bmod n$$y^d\bmod n$. La correctitud de RSA garantiza que esta salida cumple $x^e\equiv y\pmod n$$x^e\equiv y\pmod n$.

Supongamos ahora que una falla cambia únicamente $x_p$$x_p$ por $\widehat{x}_p\not\equiv x_p\pmod p$$\widehat{x}_p\not\equiv x_p\pmod p$. El residuo $x_q$$x_q$, los componentes de la clave y la operación de recombinación permanecen correctos. La implementación aplica la misma fórmula, pero con el residuo alterado:

$$ \widehat{h}=q_{\mathrm{inv}}(\widehat{x}_p-x_q)\bmod p, \qquad \boxed{\widehat{x}=x_q+q\widehat{h}.} $$$$ \widehat{h}=q_{\mathrm{inv}}(\widehat{x}_p-x_q)\bmod p, \qquad \boxed{\widehat{x}=x_q+q\widehat{h}.} $$

Esta fórmula permite ver exactamente qué conserva y qué altera la falla. Módulo $q$$q$, el término $q\widehat{h}$$q\widehat{h}$ desaparece y queda

$$ \widehat{x}\equiv x_q\pmod q. $$$$ \widehat{x}\equiv x_q\pmod q. $$

Módulo $p$$p$, en cambio, se reconstruye el residuo defectuoso:

$$ \widehat{x} \equiv x_q+q\,q_{\mathrm{inv}}(\widehat{x}_p-x_q) \equiv x_q+(\widehat{x}_p-x_q) =\widehat{x}_p\pmod p. $$$$ \widehat{x} \equiv x_q+q\,q_{\mathrm{inv}}(\widehat{x}_p-x_q) \equiv x_q+(\widehat{x}_p-x_q) =\widehat{x}_p\pmod p. $$

El atacante conoce únicamente $pk=(n,e)$$pk=(n,e)$, la entrada $y$$y$ y la salida defectuosa $\widehat{x}$$\widehat{x}$. No necesita conocer los residuos ni los factores para aprovechar estas congruencias. La rama módulo $q$$q$ sigue correcta, por lo que

$$ \widehat{x}^{\,e}\equiv x_q^e\equiv x^e\equiv y\pmod q. $$$$ \widehat{x}^{\,e}\equiv x_q^e\equiv x^e\equiv y\pmod q. $$

En la rama módulo $p$$p$, elevar a $e$$e$ es una permutación porque $\gcd(e,p-1)=1$$\gcd(e,p-1)=1$; su inversa es elevar a $d_p$$d_p$, pues $ed_p\equiv1\pmod{p-1}$$ed_p\equiv1\pmod{p-1}$. Por ello, dos residuos distintos no pueden producir la misma potencia $e$$e$-ésima. Como $\widehat{x}_p\not\equiv x_p\pmod p$$\widehat{x}_p\not\equiv x_p\pmod p$, tenemos

$$ \widehat{x}^{\,e}\equiv\widehat{x}_p^{\,e} \not\equiv x_p^e\equiv y\pmod p. $$$$ \widehat{x}^{\,e}\equiv\widehat{x}_p^{\,e} \not\equiv x_p^e\equiv y\pmod p. $$

La diferencia $r=\widehat{x}^{\,e}-y$$r=\widehat{x}^{\,e}-y$ es, por tanto, divisible por $q$$q$ pero no por $p$$p$. Como los únicos factores primos de $n$$n$ son $p$$p$ y $q$$q$,

$$ \boxed{\gcd(r,n)=q.} $$$$ \boxed{\gcd(r,n)=q.} $$

El código evita construir la potencia entera $\widehat{x}^{\,e}$$\widehat{x}^{\,e}$: calcula primero la potencia modular y reduce también la diferencia,

$$ r_0=\bigl((\widehat{x}^{\,e}\bmod n)-y\bigr)\bmod n. $$$$ r_0=\bigl((\widehat{x}^{\,e}\bmod n)-y\bigr)\bmod n. $$

Como $r_0\equiv r\pmod n$$r_0\equiv r\pmod n$, ambos valores difieren en un múltiplo de $n$$n$. Sumar o restar un múltiplo de $n$$n$ no cambia el MCD con $n$$n$, de modo que

$$ \gcd(r_0,n)=\gcd(r,n)=q. $$$$ \gcd(r_0,n)=\gcd(r,n)=q. $$

Una vez obtenido $q$$q$, el atacante reconstruye la clave privada mediante

$$ p=\frac{n}{q},\qquad \varphi(n)=(p-1)(q-1),\qquad d=e^{-1}\bmod\varphi(n). $$$$ p=\frac{n}{q},\qquad \varphi(n)=(p-1)(q-1),\qquad d=e^{-1}\bmod\varphi(n). $$

Una sola salida defectuosa permite recuperar la clave bajo estas hipótesis. La salida correcta $x$$x$ se usa para justificar el ataque, pero el atacante no necesita conocerla. El factor obtenido corresponde a la rama que permaneció correcta.

Simulamos la falla mediante $\widehat{x}_p=(x_p+1)\bmod p$$\widehat{x}_p=(x_p+1)\bmod p$ y conservamos $x_q$$x_q$. La función que representa al dispositivo usa la privada; la función del atacante recibe únicamente $pk,y,\widehat{x}$$pk,y,\widehat{x}$.

```
InvertirRSACRTConFalla(sk_crt, y)
    validar 0 ≤ y < n
    x_p ← PotenciaModular(y, d_p, p)
    x_q ← PotenciaModular(y, d_q, q)
    x_p_alterado ← (x_p + 1) mod p
    devolver RecombinarRSACRT(x_p_alterado, x_q, sk_crt)

FactorDesdeFalla(pk, y, x_alterado)
    validar que y y x_alterado pertenecen a Z_n
    r ← (EvaluarRSA(pk, x_alterado) - y) mod n
    f ← MCD(r, n)
    si 1 < f < n: devolver f
    devolver ⊥
```

El experimento recupera $q$$q$, reconstruye la clave con 6.1 y la utiliza para invertir también el reto anterior del bloque 4.

```
    def invertir_rsa_crt_con_falla(sk_crt, y):
        """Dispositivo de laboratorio: altera únicamente el residuo módulo p."""
        validar_elemento_rsa(y, sk_crt["n"])
        xp = potencia_modular(y, sk_crt["dp"], sk_crt["p"])
        xq = potencia_modular(y, sk_crt["dq"], sk_crt["q"])
        xp_alterado = (xp + 1) % sk_crt["p"]
        return recombinar_rsa_crt(xp_alterado, xq, sk_crt)

    def factor_desde_falla(pk, y, x_alterado):
        """Atacante: usa exclusivamente pk, la entrada y y la salida defectuosa."""
        validar_elemento_rsa(y, pk["n"])
        validar_elemento_rsa(x_alterado, pk["n"])
        diferencia = (evaluar_rsa(pk, x_alterado) - y) % pk["n"]
        factor = mcd(diferencia, pk["n"])
        if 1 < factor < pk["n"]:
            return factor
        return None
```

```
    pk_rsa, sk_rsa = generar_claves_rsa(BITS_PRIMO, EXPONENTE_PUBLICO, RONDAS_MR)
    sk_crt=preparar_clave_crt(sk_rsa)
```

```
    Primo p generado.
    Primo q generado.
```

```
    y_falla = secrets.randbelow(pk_rsa["n"])
    x_alterado = invertir_rsa_crt_con_falla(sk_crt, y_falla)
    validar_elemento_rsa(x_alterado, pk_rsa["n"])
    factor_recuperado = factor_desde_falla(pk_rsa, y_falla, x_alterado)
```

```
    assert factor_recuperado is not None
    assert factor_recuperado == sk_rsa["q"]

    sk_recuperada = recuperar_clave_desde_factor(pk_rsa, factor_recuperado)

    x_rsa=secrets.randbelow(pk_rsa["n"])
    y_rsa=evaluar_rsa(pk_rsa,x_rsa)
    assert invertir_rsa(sk_recuperada, y_rsa) == x_rsa
    print("Factor no trivial recuperado:", 1 < factor_recuperado < pk_rsa["n"])
    print("Bits del factor:", factor_recuperado.bit_length())
    print("La clave recuperada invierte también el reto RSA anterior.")
```

```
    Factor no trivial recuperado: True
    Bits del factor: 2048
    La clave recuperada invierte también el reto RSA anterior.
```

6.6. Verificar antes de entregar

La implementación puede detectar la salida defectuosa antes de devolverla. Si $z$$z$ es el resultado calculado, debe cumplir

$$ z^e\bmod n=y. $$$$ z^e\bmod n=y. $$

```
InvertirRSACRTVerificado(sk_crt, y)
    z ← resultado de la operación privada CRT
    si PotenciaModular(z, e, n) ≠ y:
        devolver ⊥
    devolver z
```

La prueba acepta la inversión normal y rechaza la falla simulada. También comprueba que una salida correcta produce un MCD igual a $n$$n$, por lo que no revela un factor no trivial.

La verificación usa el exponente público $65537$$65537$, cuyo reducido número de bits activos hace económica esta comprobación. Su eficacia presupone que la verificación y el control de salida no sean alterados.

```
    def invertir_rsa_crt_verificado(sk_crt, y, simular_falla=False):
        """Verifica el resultado antes de devolverlo; la bandera es solo del laboratorio."""
        if simular_falla:
            resultado = invertir_rsa_crt_con_falla(sk_crt, y)
        else:
            resultado = invertir_rsa_crt(sk_crt, y)
        if potencia_modular(resultado, sk_crt["e"], sk_crt["n"]) != y:
            raise ValueError("La verificación RSA falló; no se entrega el resultado.")
        return resultado
```

```
    assert invertir_rsa_crt_verificado(sk_crt, y_rsa) == x_rsa
    assert factor_desde_falla(pk_rsa, y_rsa, x_rsa) is None
    try:
        invertir_rsa_crt_verificado(sk_crt, y_falla, simular_falla=True)
    except ValueError as error:
        print("Falla detectada:", error)
```

```
    Falla detectada: La verificación RSA falló; no se entrega el resultado.
```

7. RSA-OAEP

7.1. Cifrar con aleatoriedad

El ataque al PIN mostró que el determinismo de RSA permite comparar cifrados con imágenes de mensajes candidatos. OAEP prepara el mensaje con una semilla aleatoria antes de aplicar RSA, de modo que un mismo mensaje puede producir muchos cifrados.

Distinguimos ahora el mensaje en bytes $M$$M$ del entero que recibe la permutación RSA. El esquema completo tiene un cifrado aleatorio $E$$E$ y un descifrado $D$$D$ que satisface

$$ D(sk,E(pk,M;r))=M $$$$ D(sk,E(pk,M;r))=M $$

para mensajes válidos y semillas $r$$r$. Ante un cifrado que no supera las comprobaciones, el descifrado devuelve rechazo.

7.2. OAEP: preparar y enmascarar

Sea $k$$k$ la longitud del módulo $n$$n$ en bytes, $h$$h$ la longitud de salida del hash y $L$$L$ una etiqueta pública acordada. Usamos SHA-256, por lo que $h=32$$h=32$ bytes. El mensaje $M$$M$ debe satisfacer $|M|\le k-2h-2$$|M|\le k-2h-2$.

Usamos $\parallel$$\parallel$ para concatenar y $\oplus$$\oplus$ para aplicar XOR entre cadenas de igual longitud. Todas las longitudes se expresan en bytes.

Primero formamos el bloque

$$ DB=\operatorname{Hash}(L)\parallel PS\parallel\texttt{0x01}\parallel M, $$$$ DB=\operatorname{Hash}(L)\parallel PS\parallel\texttt{0x01}\parallel M, $$

donde $PS$$PS$ contiene $k-|M|-2h-2$$k-|M|-2h-2$ bytes cero. Así, $DB$$DB$ tiene $k-h-1$$k-h-1$ bytes.

Elegimos una semilla uniforme $r$$r$ de $h$$h$ bytes y aplicamos las máscaras en este orden:

$$ DB'=DB\oplus W(r),\qquad r'=r\oplus H(DB'). $$$$ DB'=DB\oplus W(r),\qquad r'=r\oplus H(DB'). $$

Las funciones utilizadas se definen así:

Función	Implementación	Bytes de salida	
$\operatorname{Hash}(L)$$\operatorname{Hash}(L)$	$\operatorname{SHA256}(L)$$\operatorname{SHA256}(L)$	$h=32$$h=32$	Ca
$W(r)$$W(r)$	$\operatorname{MGF1}_{\mathrm{SHA256}}(r,k-h-1)$$\operatorname{MGF1}_{\mathrm{SHA256}}(r,k-h-1)$	$k-h-1$$k-h-1$	Ge
$H(DB')$$H(DB')$	$\operatorname{MGF1}_{\mathrm{SHA256}}(DB',h)$$\operatorname{MGF1}_{\mathrm{SHA256}}(DB',h)$	$h=32$$h=32$	Ge

MGF1 genera una máscara de la longitud solicitada. Aplica SHA-256 a la entrada seguida de un contador de cuatro bytes, en orden *big-endian* (byte más significativo primero), comenzando en cero. Concatena los resultados y toma los primeros bytes necesarios. Como $H$$H$ solicita solo $32$$32$ bytes, basta el primer resultado:

$$ H(DB')=\operatorname{SHA256}(DB'\parallel\texttt{0x00000000}), $$$$ H(DB')=\operatorname{SHA256}(DB'\parallel\texttt{0x00000000}), $$

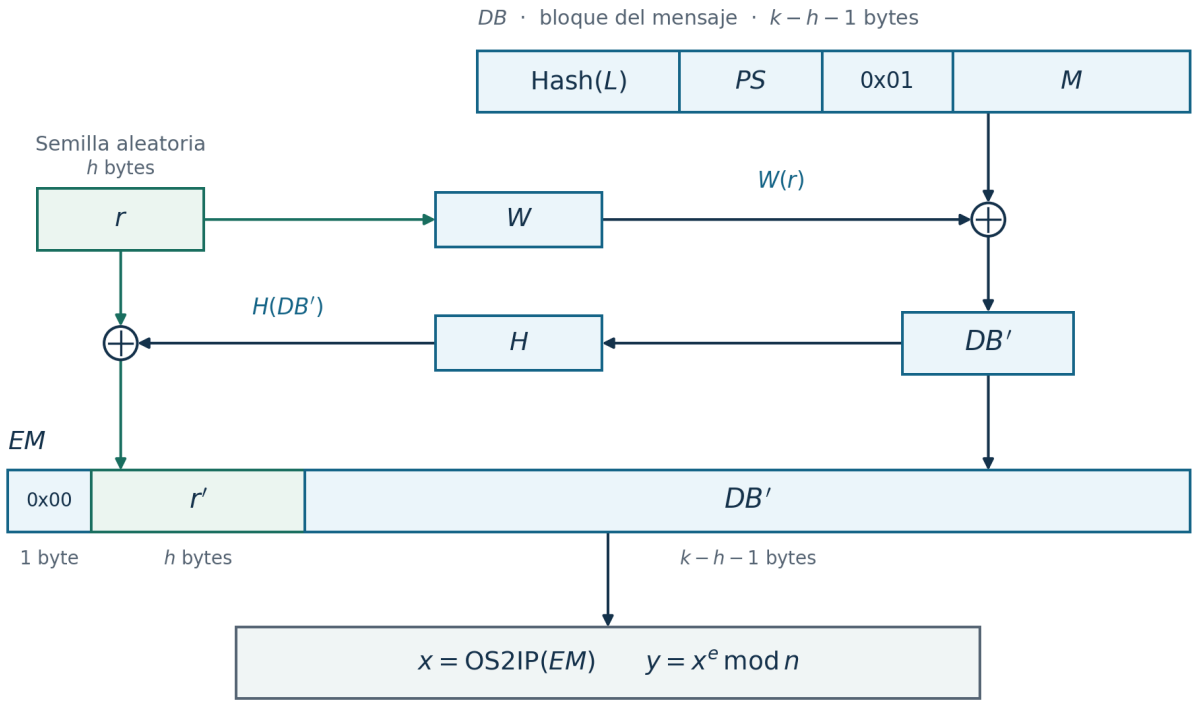
donde el sufijo representa cuatro bytes cero. Por su parte, $\operatorname{Hash}(L)$$\operatorname{Hash}(L)$ aplica SHA-256 directamente a $L$$L$, sin añadir un contador.

Finalmente, formamos el bloque codificado

$$ EM=\texttt{0x00}\parallel r'\parallel DB', $$$$ EM=\texttt{0x00}\parallel r'\parallel DB', $$

que ocupa exactamente $1+h+(k-h-1)=k$$1+h+(k-h-1)=k$ bytes. El diagrama resume la construcción y su conexión con RSA:

OAEP: dos enmascaramientos antes de RSA



Para cifrar con la clave pública $(n,e)$$(n,e)$, convertimos $EM$$EM$ a un entero, aplicamos RSA y representamos el resultado mediante $k$$k$ bytes:

$$ x=\operatorname{OS2IP}(EM),\qquad y=x^e\bmod n,\qquad C=\operatorname{I2OSP}(y,k). $$$$ x=\operatorname{OS2IP}(EM),\qquad y=x^e\bmod n,\qquad C=\operatorname{I2OSP}(y,k). $$

OS2IP interpreta una cadena de bytes como un entero, con el byte más significativo primero. **I2OSP** realiza la conversión inversa y completa con bytes cero a la izquierda hasta alcanzar la longitud indicada. El byte inicial $\texttt{0x00}$$\texttt{0x00}$ de $EM$$EM$ garantiza que $x<n$$x<n$. Al descifrar, después de recuperar $x$$x$ mediante RSA, se calcula $EM=\operatorname{I2OSP}(x,k)$$EM=\operatorname{I2OSP}(x,k)$ para reconstruir el bloque completo, incluido su byte cero inicial, antes de deshacer las máscaras.

7.3. Deshacer las máscaras

Tras invertir RSA y representar la salida en $k$$k$ bytes, se separan el prefijo, $r'$$r'$ y $DB'$$DB'$. La recuperación invierte el orden anterior:

$$ r=r'\oplus H(DB'),\qquad DB=DB'\oplus W(r). $$$$ r=r'\oplus H(DB'),\qquad DB=DB'\oplus W(r). $$

Primero se recupera la semilla porque $DB'$$DB'$ ya está disponible. Después se obtiene $DB$$DB$. En cada paso, aplicar otra vez la misma máscara XOR cancela su efecto.

El receptor comprueba el prefijo cero, $\operatorname{Hash}(L)$$\operatorname{Hash}(L)$, el relleno y el separador `0x01` antes de devolver $M$$M$. Un rechazo OAEP no entrega el entero interno obtenido al invertir RSA; por sí solo, no proporciona la salida que requiere el ataque por fallas.

7.4. Seudocódigo de cifrado y descifrado

El procedimiento reúne las conversiones, las máscaras y las comprobaciones anteriores. Emisor y receptor usan el mismo hash, la misma etiqueta $L$$L$ y un módulo cuya longitud satisface $k\ge2h+2$$k\ge2h+2$.

```
CifrarOAEP(pk, M, L)
    k ← longitud del módulo RSA en bytes
    si |M| > k - 2h - 2: rechazar
    DB ← Hash(L) || ceros de relleno || 0x01 || M
    r ←$ cadena de h bytes
    DB' ← DB XOR W(r)
    r' ← r XOR H(DB')
    x ← OS2IP(0x00 || r' || DB')
    devolver I2OSP(F_RSA(pk, x), k)

DescifrarOAEP(sk, C, L)
    k ← longitud del módulo RSA en bytes
    si |C| ≠ k o OS2IP(C) ≥ n: devolver ⊥
    EM ← I2OSP(I_RSA(sk, OS2IP(C)), k)
    separar EM en (prefijo, r', DB')
    r ← r' XOR H(DB')
    DB ← DB' XOR W(r)
    si prefijo ≠ 0x00 o DB no tiene el formato esperado: devolver ⊥
    devolver el mensaje M contenido en DB
```

Usaremos PyCryptodome para ejecutar este esquema y verificar el formato completo al descifrar.

7.5. Ejecutar OAEP con nuestra clave

`RSA.construct` convierte los componentes generados en 3.5 en un objeto de PyCryptodome. Se conserva la misma clave RSA. Fijamos SHA-256, con MGF1 basado en ese hash, y la etiqueta vacía `L = b""`.

El experimento cifra dos veces el mismo mensaje con semillas nuevas. Los cifrados difieren salvo que las semillas coincidan; ambos descifrados deben recuperar los mismos bytes. Cada cifrado ocupa $k$$k$ bytes.

```
%pip -q install pycryptodome==3.23.0
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 34.1 MB/s eta 0:00:00
```

```

```

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP, AES
from Crypto.Hash import SHA256

clave_privada_lib = RSA.construct(
    (sk_rsa["n"], sk_rsa["e"], sk_rsa["d"], sk_rsa["p"], sk_rsa["q"])
)
clave_publica_lib = clave_privada_lib.publickey()
k_rsa = clave_publica_lib.size_in_bytes()
h_oaep = SHA256.digest_size
limite_oaep = k_rsa - 2 * h_oaep - 2

print("Bytes del módulo RSA:", k_rsa)
print("Capacidad de RSA-OAEP:", limite_oaep, "bytes")
```

```
Bytes del módulo RSA: 512
Capacidad de RSA-OAEP: 446 bytes
```

```
mensaje_oaep = "Un mensaje corto protegido con RSA-OAEP.".encode("utf-8")

# La pública permite cifrar. Cada llamada usa aleatoriedad nueva.
cifrador_oaep = PKCS1_OAEP.new(clave_publica_lib, hashAlgo=SHA256)
cifrado_1 = cifrador_oaep.encrypt(mensaje_oaep)
cifrado_2 = cifrador_oaep.encrypt(mensaje_oaep)
print(cifrado_1)
print(cifrado_2)
# La privada permite recuperar el mensaje.
descifrador_oaep = PKCS1_OAEP.new(clave_privada_lib, hashAlgo=SHA256)
recuperado_1 = descifrador_oaep.decrypt(cifrado_1)
recuperado_2 = descifrador_oaep.decrypt(cifrado_2)

print("Los cifrados son distintos:", cifrado_1 != cifrado_2)
print("Primer descifrado:", recuperado_1.decode("utf-8"))
print("Segundo descifrado:", recuperado_2.decode("utf-8"))
assert recuperado_1 == recuperado_2 == mensaje_oaep
```

```
b':\x0c\xb5\xc4\xc1G,\xd8?Zs\x86\xa0\xafsd\x1dM\n\xed*\x7f\xb6\x85\xfe\xfd\x10\x07\xfa\x02-a4>\xc9\xf2\x94\xbc(\xab\xfd\x9b\x9dQ
b')\xc8\x1b\x85\xd2/\x8dD\x1a$\x85\xfd\x81\xc1\x00\x80\x03\xab7\xcdU\xcb6\xd3#\x80\x8b\x9cQ\xc2\x01\x86\xaa\xd7@8\x88\x06Q:\x00S
Los cifrados son distintos: True
Primer descifrado: Un mensaje corto protegido con RSA-OAEP.
Segundo descifrado: Un mensaje corto protegido con RSA-OAEP.
```

7.6. Del límite de OAEP al cifrado híbrido

El relleno y las máscaras reservan $2h+2$$2h+2$ bytes, de modo que

$$ |M|\le k-2h-2. $$$$ |M|\le k-2h-2. $$

Nuestra clave ocupa $k=512$$k=512$ bytes y SHA-256 tiene $h=32$$h=32$: el máximo es **446 bytes** por mensaje OAEP. La cuenta se hace en bytes; un carácter UTF-8 puede ocupar varios.

Para enviar archivos, protegeremos un secreto aleatorio de 32 bytes con RSA-OAEP y derivaremos de él una clave AES. AES-GCM usará esa clave para cifrar los datos y comprobar su integridad. Esta combinación de cifrado de clave pública y cifrado simétrico es el **cifrado híbrido**.

8. Aplicación final: archivos cifrados con RSA-OAEP y AES-GCM

8.1. Participantes y componentes

Ahora integraremos lo estudiado en una aplicación. Alice actúa como **servidor** y conserva la clave privada. Cada cliente Bob conoce su clave pública auténtica y abre una conexión para enviar mensajes y archivos.

Cada conexión tendrá su propia clave de sesión y permanecerá abierta para realizar varios envíos. Organizamos las funciones según su propósito:

Componente	Responsabilidad
RSA-OAEP	Transportar un secreto aleatorio hasta Alice
Derivación	Obtener la clave AES de la sesión
AES-GCM	Cifrar y verificar datos y confirmaciones

Componente	Responsabilidad
TCP	Transportar paquetes delimitados por su longitud
Aplicación	Interpretar mensajes y guardar archivos verificados

Definiremos primero estas operaciones y luego las reuniremos en el servidor y los clientes.

Todo se ejecuta en `127.0.0.1`, la dirección del propio entorno de Colab. Los participantes comparten el proceso del cuaderno. Los hilos permiten atender varias conexiones, pero no aíslan las claves entre participantes.

8.2. Establecer la clave de sesión

Un mecanismo de encapsulación de claves, o **KEM**, permite obtener un secreto compartido a partir de una clave pública y una encapsulación. Aquí implementamos esa interfaz cifrando un secreto aleatorio con RSA-OAEP.

Una vez abierta la conexión:

1. Bob genera un secreto $S$$S$ de 32 bytes aleatorios y envía $C_S=\operatorname{OAEP.Enc}(pk_{\mathrm{Alice}},S)$$C_S=\operatorname{OAEP.Enc}(pk_{\mathrm{Alice}},S)$.
2. Alice descifra $C_S$$C_S$, obtiene $S$$S$ y devuelve una sal aleatoria pública de 16 bytes.
3. Ambos calculan $K=\operatorname{HKDF}(S,\mathrm{sal},\mathrm{contexto})$$K=\operatorname{HKDF}(S,\mathrm{sal},\mathrm{contexto})$, con SHA-256 y una salida de 32 bytes.
4. Alice envía una confirmación cifrada con $K$$K$. Bob la verifica antes de enviar datos.

HKDF deriva claves a partir de un secreto mediante HMAC, una construcción basada en un hash y una clave. Usamos el contexto fijo `b"Colab01/sesion"` para identificar este uso de la clave derivada. La misma entrada, sal y contexto producen la misma clave AES-256 en ambos extremos.

La sal fresca permite variar la derivación incluso si se vuelve a recibir una encapsulación anterior. $S$$S$ y $K$$K$ son secretos; la sal es pública. $S$$S$ tampoco es la semilla interna de OAEP: es el mensaje que OAEP protege.

```
encapsular(pk)                  → (encapsulacion, secreto)
desencapsular(sk, encapsulacion)  → secreto
```

Las siguientes funciones implementan esta interfaz y la derivación. RSA-OAEP se usa al establecer la sesión; los envíos posteriores usan AES-GCM.

```
import socket
from pathlib import Path
from tempfile import mkdtemp
from threading import Thread, Event
from Crypto.Protocol.KDF import HKDF

HOST = "127.0.0.1"
MAX_PAQUETE = 8 * 1024 * 1024  # 8 MiB, incluidos los campos del paquete.

def encapsular_oaep(pk):
    secreto = secrets.token_bytes(32)
    publica = RSA.construct((pk["n"], pk["e"]))
    oaep = PKCS1_OAEP.new(publica, hashAlgo=SHA256)
    return oaep.encrypt(secreto), secreto

def desencapsular_oaep(sk, encapsulacion):
    privada = RSA.construct((sk["n"], sk["e"], sk["d"], sk["p"], sk["q"]))
    oaep = PKCS1_OAEP.new(privada, hashAlgo=SHA256)
    secreto = oaep.decrypt(encapsulacion)
    if len(secreto) != 32:
        raise ValueError("Se esperaba un secreto de 32 bytes.")
    return secreto

def derivar_clave_sesion(secreto, sal):
    if len(sal) != 16:
        raise ValueError("La sal de sesión debe tener 16 bytes.")
    return HKDF(secreto, 32, sal, SHA256, context=b"Colab01/sesion")
```

8.3. Cifrar varios envíos con AES-GCM

AES-GCM cifra el contenido $M$$M$ y produce un cifrado $C$$C$ junto con una etiqueta $T$$T$ que permite verificar su integridad. Usaremos un nonce $\nu$$\nu$ de 12 bytes y una etiqueta de 16 bytes:

$$ P=\nu\parallel T\parallel C. $$$$ P=\nu\parallel T\parallel C. $$

Un nonce no debe repetirse con la misma clave. Como Alice y Bob comparten $K$$K$, asignamos contadores distintos a cada dirección:

Emisor	Contadores
Bob, para mensajes y archivos	0, 2, 4, 6, …
Alice, para confirmaciones	1, 3, 5, 7, …

El nonce es el contador representado en 12 bytes. Cada receptor comprueba el siguiente contador esperado, por lo que también rechaza paquetes repetidos o fuera de secuencia dentro de la sesión.

`encrypt_and_digest` devuelve el cifrado y la etiqueta. `decrypt_and_verify` solo devuelve los datos si la etiqueta es válida. El nonce se transmite en claro; su propósito es asegurar la unicidad, no aportar un secreto.

```
    def cifrar_simetrico(clave, datos, contador):
        nonce = contador.to_bytes(12, "big")
        aes = AES.new(clave, AES.MODE_GCM, nonce=nonce, mac_len=16)
        cifrado, etiqueta = aes.encrypt_and_digest(datos)
        return nonce + etiqueta + cifrado

    def descifrar_simetrico(clave, paquete, contador_esperado):
        if len(paquete) < 28:
            raise ValueError("El paquete cifrado está incompleto.")
        nonce = paquete[:12]
        etiqueta = paquete[12:28]
        cifrado = paquete[28:]
        if nonce != contador_esperado.to_bytes(12, "big"):
            raise ValueError("El contador recibido no es el esperado.")
        aes = AES.new(clave, AES.MODE_GCM, nonce=nonce, mac_len=16)
        return aes.decrypt_and_verify(cifrado, etiqueta)
```

8.4. Enviar varios paquetes por la misma conexión TCP

TCP transporta una secuencia de bytes y no conserva los límites entre los envíos. Una llamada a `recv` puede devolver solo una parte de lo que esperamos. Por ello, precedemos cada paquete con su longitud:

$$ \mathrm{envio}=\mathrm{longitud}_{4\ \mathrm{bytes}}\parallel\mathrm{paquete}. $$$$ \mathrm{envio}=\mathrm{longitud}_{4\ \mathrm{bytes}}\parallel\mathrm{paquete}. $$

El receptor completa primero los cuatro bytes del prefijo y después lee la cantidad indicada. `"big"` coloca primero el byte más significativo.

Función	Propósito
`recibir_exacto`	Acumular bytes hasta completar la cantidad solicitada
`enviar_paquete`	Añadir el prefijo y enviar todo mediante `sendall`
`recibir_paquete`	Leer el prefijo y luego el cuerpo

`recv` devuelve `b""` al cerrarse la conexión. Si el cierre ocurre antes del siguiente prefijo, devolvemos `None`; si ocurre durante un paquete, informamos un error.

El cuerpo admite hasta 8 MiB, con $1\ \mathrm{MiB}=2^{20}$$1\ \mathrm{MiB}=2^{20}$ bytes. El prefijo queda fuera de ese límite. Los archivos se procesan completos en memoria y la longitud se transmite en claro.

```
    def recibir_exacto(conexion, cantidad):
        datos = bytearray()
        while len(datos) < cantidad:
            parte = conexion.recv(min(65536, cantidad - len(datos)))
            if parte == b"":
                if len(datos) == 0:
                    return None
                raise ConnectionError("La conexión terminó a mitad del paquete.")
            datos.extend(parte)
        return bytes(datos)

    def enviar_paquete(conexion, paquete):
        if not 0 < len(paquete) <= MAX_PAQUETE:
```

```
            raise ValueError("La longitud del paquete está fuera del límite.")
        prefijo = len(paquete).to_bytes(4, "big")
        conexion.sendall(prefijo + paquete)

    def recibir_paquete(conexion):
        prefijo = recibir_exacto(conexion, 4)
        if prefijo is None:
            return None
        cantidad = int.from_bytes(prefijo, "big")
        if not 0 < cantidad <= MAX_PAQUETE:
            raise ValueError("La longitud anunciada está fuera del límite.")
        paquete = recibir_exacto(conexion, cantidad)
        if paquete is None:
            raise ConnectionError("Falta el cuerpo del paquete anunciado.")
        return paquete
```

8.5. Conservar el estado de cada sesión

Para reutilizar una conexión, cada participante debe conservar su clave y sus contadores. Los reunimos en un diccionario:

Entrada	Significado
`conexion`	Socket de esta conexión
`clave`	Clave AES compartida en esta sesión
`envio`	Contador del siguiente envío local
`recepcion`	Contador esperado al recibir

`enviar_cifrado` comprueba primero que los datos, junto con los 28 bytes del nonce y la etiqueta, no superen el límite del paquete. Si lo superan, rechaza el envío sin cifrar ni modificar el contador. En caso contrario, cifra con el contador disponible, lo incrementa en dos y envía el paquete. Si falla el envío por un error de red, cierra la conexión; el contador no retrocede porque parte del paquete podría haberse transmitido.

`recibir_cifrado` verifica el contador y la etiqueta; solo si ambas comprobaciones pasan, incrementa el contador de recepción.

Cada sesión se usa secuencialmente y mantiene su propio estado. Sus contadores no se reinician mientras se reutilice la clave.

```
    def crear_sesion(conexion, clave, primer_envio):
        return {"conexion": conexion, "clave": clave,
                "envio": primer_envio, "recepcion": 1 - primer_envio}

    def enviar_cifrado(sesion, datos):
        if len(datos) + 28 > MAX_PAQUETE:
            raise ValueError("Los datos, el nonce y la etiqueta superan el límite del paquete.")
        paquete = cifrar_simetrico(sesion["clave"], datos, sesion["envio"])
        sesion["envio"] += 2
        try:
            enviar_paquete(sesion["conexion"], paquete)
        except OSError:
            sesion["conexion"].close()
            raise

    def recibir_cifrado(sesion):
        paquete = recibir_paquete(sesion["conexion"])
        if paquete is None:
            return None
        datos = descifrar_simetrico(sesion["clave"], paquete, sesion["recepcion"])
        sesion["recepcion"] += 2
        return datos
```

8.6. Representar mensajes y archivos

Antes de cifrar, representamos el contenido de la aplicación como

$$ M=\mathrm{nombre\ en\ UTF8}\parallel\texttt{0x00}\parallel\mathrm{datos}. $$$$ M=\mathrm{nombre\ en\ UTF8}\parallel\texttt{0x00}\parallel\mathrm{datos}. $$

Un nombre vacío indica un mensaje; uno no vacío indica un archivo. El primer byte cero separa ambos campos. Los datos pueden contener otros ceros porque `split(b"\x00", 1)` solo divide en la primera aparición.

Contenido	Representación
Mensaje	`b"\x00" + texto.encode("utf-8")`
Archivo	`ruta.name.encode("utf-8") + b"\x00" + ruta.read_bytes()`

Se cifra todo $M$$M$, incluido el nombre. Alice verifica y descifra antes de interpretar los campos. Solo admite nombres simples, no rutas. Cada conexión tiene una carpeta propia y cada archivo recibe un número para conservar por separado los envíos con el mismo nombre.

El recorrido completo es: formar $M$$M$, cifrarlo en un paquete $P$$P$ y añadir la longitud para transportarlo por TCP. Al recibir, se invierte ese orden.

8.7. Alice atiende una conexión

`atender_cliente` reúne los componentes para una conexión: recupera el secreto, envía la sal, deriva la clave y confirma el establecimiento. Después recibe contenidos en un ciclo, los verifica, procesa el mensaje o archivo y devuelve una confirmación cifrada.

La función `desencapsular` se recibe como argumento, de modo que el manejador utiliza la interfaz de 8.2. Si el cliente cierra, la recepción devuelve `None` y termina el ciclo. Ante un paquete inválido o un error de entrada/salida, se cierra esa conexión. El bloque `with conexion` garantiza el cierre del socket al salir.

```
    def atender_cliente(conexion, sk, desencapsular, carpeta):
        with conexion:
            try:
                encapsulacion = recibir_paquete(conexion)
                if encapsulacion is None:
                    return
                secreto = desencapsular(sk, encapsulacion)
                sal = secrets.token_bytes(16)
                enviar_paquete(conexion, sal)
                clave = derivar_clave_sesion(secreto, sal)
                sesion = crear_sesion(conexion, clave, primer_envio=1)
                enviar_cifrado(sesion, b"Sesion establecida")

                numero_archivo = 1
                while True:
                    contenido = recibir_cifrado(sesion)
                    if contenido is None:
                        break
                    nombre_bytes, datos = contenido.split(b"\x00", 1)
                    nombre = nombre_bytes.decode("utf-8")
                    if nombre == "":
                        print("Alice recibió texto:", datos.decode("utf-8"))
                        respuesta = "Texto recibido"
                    else:
                        if nombre != Path(nombre).name or nombre in (".", "..") or "\\" in nombre:
                            raise ValueError("Se esperaba un nombre de archivo, no una ruta.")
                        destino = carpeta / f"{numero_archivo:02d}_{nombre}"
                        destino.write_bytes(datos)
                        numero_archivo += 1
                        print("Alice guardó:", destino)
                        respuesta = f"Archivo recibido: {nombre} ({len(datos)} bytes)"
                    enviar_cifrado(sesion, respuesta.encode("utf-8"))
            except (ValueError, OSError) as error:
                print("Alice cerró una conexión:", error)
```

8.8. Alice acepta varios clientes

El servidor separa la aceptación de conexiones de su atención. `accept` obtiene una nueva conexión; se crea una carpeta para ella y un hilo que ejecuta `atender_cliente`. Mientras ese hilo espera datos, el ciclo de aceptación puede recibir otros clientes.

El diccionario `servidor` guarda el socket de escucha, las conexiones, los hilos y una señal de parada `Event`. El ciclo consulta esa señal mediante `is_set()`.

La espera de `accept` vence cada medio segundo para volver a consultar la señal. Ese tiempo límite afecta al socket de escucha; las conexiones de los clientes permanecen abiertas mientras esperan datos.

```
    def aceptar_clientes(servidor, sk, desencapsular):
        while not servidor["detener"].is_set():
```

```
            try:
                conexion, direccion = servidor["socket"].accept()
            except socket.timeout:
                continue
            except OSError:
                break
            conexion.settimeout(None)
            carpeta = Path(mkdtemp(prefix="cliente_", dir=servidor["carpeta"]))
            servidor["conexiones"].append(conexion)
            hilo = Thread(target=atender_cliente,
                          args=(conexion, sk, desencapsular, carpeta), daemon=True)
            servidor["hilos"].append(hilo)
            hilo.start()
            print("Alice aceptó a un cliente en:", direccion)
```

Inicio y cierre del servidor. `bind` asocia el socket a una dirección. El puerto `0` solicita uno disponible, que se consulta con `getsockname`. `listen` prepara la escucha antes de iniciar el hilo de aceptación.

Para detener el servidor se activa la señal, se espera el fin del ciclo de aceptación y se cierran las conexiones. `shutdown` interrumpe las recepciones pendientes y `join` espera la terminación de los hilos. Aunque `daemon=True` permite que el proceso termine con hilos activos, en el ejemplo realizaremos el cierre explícito.

```
    def iniciar_servidor(sk, desencapsular, carpeta):
        carpeta = Path(carpeta)
        carpeta.mkdir(parents=True, exist_ok=True)
        escucha = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        escucha.bind((HOST, 0))
        escucha.listen()
        escucha.settimeout(0.5)
        servidor = {"socket": escucha, "direccion": escucha.getsockname(),
                    "carpeta": carpeta, "detener": Event(), "conexiones": [], "hilos": []}
        hilo = Thread(target=aceptar_clientes,
                      args=(servidor, sk, desencapsular), daemon=True)
        servidor["hilo_aceptacion"] = hilo
        hilo.start()
        return servidor

    def detener_servidor(servidor):
        servidor["detener"].set()
        servidor["hilo_aceptacion"].join()
        servidor["socket"].close()
        for conexion in servidor["conexiones"]:
            try:
                conexion.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass  # El cliente puede haber cerrado ya su conexión.
            conexion.close()
        for hilo in servidor["hilos"]:
            hilo.join()
        print("Servidor detenido; conexiones e hilos cerrados.")
```

8.9. Bob se conecta y envía

`conectar_cliente` abre un socket, encapsula el secreto con la pública auténtica de Alice y lo envía. Al recibir la sal, deriva la clave y verifica la confirmación cifrada de Alice.

Si todo termina correctamente, devuelve el diccionario de sesión para reutilizar la conexión. Si ocurre un error, cierra el socket.

```
    def conectar_cliente(direccion, pk, encapsular):
        conexion = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            conexion.connect(direccion)
            encapsulacion, secreto = encapsular(pk)
            enviar_paquete(conexion, encapsulacion)
            sal = recibir_paquete(conexion)
            if sal is None:
                raise ConnectionError("Alice cerró durante el establecimiento.")
            clave = derivar_clave_sesion(secreto, sal)
            sesion = crear_sesion(conexion, clave, primer_envio=0)
            if recibir_cifrado(sesion) != b"Sesion establecida":
```

```
            raise ValueError("No se recibió la confirmación de Alice.")
        return sesion
    except Exception:
        conexion.close()
        raise
```

`enviar_contenido` forma el contenido de 8.6, lo envía cifrado y espera la confirmación. `enviar_texto` aporta un nombre vacío; `enviar_archivo` aporta el nombre y los bytes leídos del archivo. La conexión permanece abierta al terminar cada envío.

Si falla el envío por un error de red, `enviar_cifrado` cierra la conexión. Si falla la recepción o la validación de la confirmación, `enviar_contenido` también la cierra y propaga la excepción. Para continuar hay que establecer otra sesión. En cambio, rechazar localmente un contenido demasiado grande conserva la conexión y sus contadores, por lo que puede intentarse después un envío de menor tamaño.

```
def enviar_contenido(sesion, nombre, datos):
    contenido = nombre.encode("utf-8") + b"\x00" + datos
    enviar_cifrado(sesion, contenido)
    try:
        respuesta = recibir_cifrado(sesion)
        if respuesta is None:
            raise ConnectionError("Alice cerró sin confirmar el envío.")
        return respuesta.decode("utf-8")
    except (ValueError, OSError):
        sesion["conexion"].close()
        raise

def enviar_texto(sesion, texto):
    return enviar_contenido(sesion, "", texto.encode("utf-8"))

def enviar_archivo(sesion, ruta):
    ruta = Path(ruta)
    return enviar_contenido(sesion, ruta.name, ruta.read_bytes())
```

8.10. Ejecutar la aplicación con dos clientes

Paso 1: iniciar a Alice. Reutilizamos la clave RSA del bloque 3. El servidor recibe la privada y la función `desencapsular_oaep`; los clientes recibirán la pública.

Los archivos se crean bajo `salida_colab01`, con una carpeta nueva de recepción en cada inicio. Si se repite la celda, primero se detiene el servidor anterior.

```
if "servidor_alice" in globals():
    detener_servidor(servidor_alice)

carpeta_laboratorio = Path("salida_colab01")
carpeta_laboratorio.mkdir(exist_ok=True)
carpeta_recibidos = Path(mkdtemp(prefix="recibidos_", dir=carpeta_laboratorio))
pk_alice = pk_rsa.copy()
sk_alice = sk_rsa
servidor_alice = iniciar_servidor(sk_alice, desencapsular_oaep, carpeta_recibidos)
print("Alice escucha en:", servidor_alice["direccion"])
```

```
Alice escucha en: ('127.0.0.1', 42387)
```

Paso 2: conectar dos clientes. `bob_1` y `bob_2` abren conexiones independientes y las mantienen activas. Comprobamos que sus claves de sesión sean distintas y enviamos un mensaje por cada conexión.

```
bob_1 = conectar_cliente(servidor_alice["direccion"], pk_alice, encapsular_oaep)
bob_2 = conectar_cliente(servidor_alice["direccion"], pk_alice, encapsular_oaep)

print("Los clientes tienen claves distintas:", bob_1["clave"] != bob_2["clave"])
print("Bob 1:", enviar_texto(bob_1, "Hola Alice, soy el primer cliente."))
print("Bob 2:", enviar_texto(bob_2, "Hola Alice, soy el segundo cliente."))
```

```
Alice aceptó a un cliente en: ('127.0.0.1', 50320)
Alice aceptó a un cliente en: ('127.0.0.1', 50322)
```

```
Los clientes tienen claves distintas: True
Alice recibió texto: Hola Alice, soy el primer cliente.
Bob 1: Texto recibido
Alice recibió texto: Hola Alice, soy el segundo cliente.
Bob 2: Texto recibido
```

Paso 3: preparar archivos. Creamos un texto que supera el límite de OAEP y un archivo binario con todos los valores de byte, incluido el cero. Estos datos permitirán comprobar que el cifrado híbrido conserva el contenido exacto de ambos tipos de archivo.

```
archivo_texto = carpeta_laboratorio / "mensaje.txt"
archivo_binario = carpeta_laboratorio / "datos.bin"
archivo_texto.write_text("Este archivo viaja protegido con AES-GCM.\n" * 300,
                         encoding="utf-8")
archivo_binario.write_bytes(bytes(range(256)) * 512)

print("Texto:", archivo_texto.stat().st_size, "bytes")
print("Binario:", archivo_binario.stat().st_size, "bytes")
print("Capacidad de un mensaje RSA-OAEP:", limite_oaep, "bytes")
```

```
Texto: 12600 bytes
Binario: 131072 bytes
Capacidad de un mensaje RSA-OAEP: 446 bytes
```

Paso 4: realizar varios envíos. Reutilizamos las dos sesiones: Bob 1 envía dos archivos y Bob 2 envía un archivo y otro mensaje. Las carpetas por conexión permiten conservar archivos con el mismo nombre.

Alice confirma cada archivo después de guardarlo. Se comparan las tres copias recibidas con sus originales, byte a byte. Los mensajes impresos por distintos hilos pueden aparecer en diferente orden.

```
print("Bob 1:", enviar_archivo(bob_1, archivo_texto))
print("Bob 2:", enviar_archivo(bob_2, archivo_texto))
print("Bob 1:", enviar_archivo(bob_1, archivo_binario))
print("Bob 2:", enviar_texto(bob_2, "Sigo usando la misma conexión."))

copias_texto = list(carpeta_recibidos.glob("cliente_*/*_mensaje.txt"))
copias_binario = list(carpeta_recibidos.glob("cliente_*/*_datos.bin"))
assert len(copias_texto) == 2 and len(copias_binario) == 1
for copia in copias_texto:
    assert copia.read_bytes() == archivo_texto.read_bytes()
assert copias_binario[0].read_bytes() == archivo_binario.read_bytes()
print("Verificación: los tres archivos recibidos coinciden con sus originales.")
```

```
Alice guardó: /content/salida_colab01/recibidos_5b2j_cdv/cliente_mw4nd9l6/01_mensaje.txt
Bob 1: Archivo recibido: mensaje.txt (12600 bytes)
Alice guardó: /content/salida_colab01/recibidos_5b2j_cdv/cliente_wgnhmei5/01_mensaje.txt
Bob 2: Archivo recibido: mensaje.txt (12600 bytes)
Alice guardó: /content/salida_colab01/recibidos_5b2j_cdv/cliente_mw4nd9l6/02_datos.bin
Bob 1: Archivo recibido: datos.bin (131072 bytes)
Alice recibió texto: Sigo usando la misma conexión.
Bob 2: Texto recibido
Verificación: los tres archivos recibidos coinciden con sus originales.
```

8.11. Alterar un paquete y observar el rechazo

Abrimos una tercera sesión, ciframos un archivo y cambiamos un bit de la etiqueta GCM. La posición 12 marca su inicio, justo después del nonce.

Alice debe rechazar el paquete antes de guardar el archivo y cerrar solo esa conexión. El experimento comprueba que el archivo no exista y que las otras dos sesiones sigan funcionando.

```
bob_prueba = conectar_cliente(servidor_alice["direccion"], pk_alice, encapsular_oaep)
contenido = b"alterado.txt\x00Este archivo no debe guardarse."
paquete = bytearray(cifrar_simetrico(bob_prueba["clave"], contenido, bob_prueba["envio"]))
paquete[12] ^= 1  # Invertir un bit de la etiqueta GCM.
enviar_paquete(bob_prueba["conexion"], bytes(paquete))

respuesta = recibir_paquete(bob_prueba["conexion"])
bob_prueba["conexion"].close()
assert respuesta is None
assert not list(carpeta_recibidos.glob("cliente_*/*_alterado.txt"))
```

```
print("El paquete alterado fue rechazado y el archivo no se guardó.")
print("Bob 1 continúa:", enviar_texto(bob_1, "Mi sesión sigue funcionando."))
print("Bob 2 continúa:", enviar_texto(bob_2, "La mía también."))
```

```
Alice aceptó a un cliente en: ('127.0.0.1', 50330)
Alice cerró una conexión: MAC check failed
El paquete alterado fue rechazado y el archivo no se guardó.
Alice recibió texto: Mi sesión sigue funcionando.
Bob 1 continúa: Texto recibido
Alice recibió texto: La mía también.
Bob 2 continúa: Texto recibido
```

8.12. Continuar, cerrar clientes y detener a Alice

Antes de ejecutar la siguiente celda, puede realizar más envíos con `enviar_texto(bob_1, "Otro mensaje")` o `enviar_archivo(bob_2, archivo_binario)` en una celda nueva. Se reutilizan las sesiones ya abiertas.

La celda de cierre termina ambos clientes y detiene el servidor. Para repetir el experimento completo, ejecute desde el paso 1 de 8.10.

```
bob_1["conexion"].close()
bob_2["conexion"].close()
detener_servidor(servidor_alice)
```